AGRICULTURAL KNOWLEDGE PLATFORM

AgroGyanGPT

Placement preparation
Technical architecture, deployed application and interview study guide

Updated 24 October 2026

Includes the public landing page, mobile remediation, Render + Supabase + Groq deployment, password registration, source-grounded retrieval, and step-by-step future publication workflow.

Local and cloud behavior are distinguished. Test counts are evidence of specific regressions, not an agricultural accuracy score.

Contents

Contents 2

How to use this report 4

1 Executive project summary 5

2 Problem statement and requirements 6

3 Technology stack and selection rationale 7

4 Architecture and request lifecycle 8

5 Repository responsibilities 9

6 Database design and storage 11

7 Farming only GR import and source management 12

8 PDF OCR and chunking 13

9 Lexical retrieval and BM25 13

10 Embeddings and vector search 14

11 Hybrid RAG and answer grounding 14

12 Multilingual retrieval and translation 15

13 Frontend and conversation workspace 16

14 Authentication security and privacy 17

15 Supporting modules and working level 18

16 API contracts and error design 19

17 Algorithms and engineering concepts 20

18 Performance and evaluation 20

19 Testing and verification 21

20 Configuration startup and maintenance 21

21 Known limitations and next engineering work 22

22 Cleanup performed 23

23 Placement introduction and resume 24

24 Interview questions and answer guidance 25

25 Demonstration and troubleshooting 27

26 Glossary and revision checklist 28

27 Evidence and references 28

28 Deployed free demonstration architecture 29

29 Public landing page and navigation model 31

30 Mobile remediation across the application 32

31 Publishing future changes and maintenance workflow 33

32 Interview revision for the hosted version 34

Appendix A Verification snapshots 35

Appendix B Endpoint inventory 37

Appendix C Request schema dictionary 39

Appendix D Complete database field dictionary 46

Appendix E Code symbol navigation 56

Appendix F File inventory and cleanup audit 68

How to use this report

AgroGyanGPT is an agricultural information application with a local-first development setup and a deployed free cloud demonstration at https://agrogyan-demo.onrender.com/. Its central feature retrieves evidence from farming references and government resolutions, attaches source citations, and supports English, Hindi, Marathi and Gujarati. The surrounding website provides farm profiles, weather integration, community discussions and prototype commerce workflows.

The project combines deterministic retrieval and source validation with pretrained models. Local development uses Ollama; the hosted demonstration uses Groq for evidence selection and translation, with Supabase PostgreSQL for account records and sessions. The active answer model selects a passage from bounded evidence; it is not trained from scratch. Some general questions use reviewed sourced entries without a model call. Read chapters 1 to 5 for the introduction, 6 to 14 for the technical deep dive, 15 to 21 for engineering evaluation, and 22 to 26 for interview practice. Chapters 28 to 32 cover the deployed architecture, new public landing page, mobile remediation, complete update workflow and additional interview questions. Appendices contain the API, schema and code inventories.

Paths are relative to agrogyan-gpt-main unless stated otherwise. Corpus counts describe the inspected knowledge snapshot. Local storage integrity/latency baselines retain their measurement dates; current hosting facts are identified separately. Never present historical local timings as cloud benchmarks. Historical latency observations are examples, not an SLA or a statistically representative benchmark. Source citations establish provenance; they do not prove that a historical GR is still applicable today. Implemented features, configuration-dependent integrations and future work are distinguished throughout.

1 Executive project summary

AgroGyanGPT helps farmers search agricultural guidance and government resolutions through multilingual chat. A FastAPI backend loads page-aware PDF text and an imported Maharashtra GR catalog. Local hybrid retrieval combines BM25 and optional persistent embedding rankings, with topic, state, intent and year constraints. The free hosted profile uses BM25 and Groq evidence selection instead of running Ollama. Deterministic checks validate selected evidence. The API returns answer text, original wording, source links and warnings. A public landing page introduces the product; a dedicated chat page organizes separate browser-local conversations.

Item	Verified project state
Main database	Local SQLite agrogyan.db; hosted Supabase PostgreSQL through SQLAlchemy
Application tables	16 ORM tables; hosted opaque sessions use a separate sessions table
Unique Maharashtra farming GRs	662 identifiers
Imported language records	1324 English and Marathi variants
Active Maharashtra GR content	2048 pages and 4239 chunks
Active directly ingested Gujarat PDF chunks	45 from 3 included PDFs
Curated farming references	18 JSON entries plus a separate soil reference
Active retrieval passages	4303
Embedding dimension	768
Persisted vectors	4302 distinct embeddings mapped to 4303 chunks
Search	Local BM25 + NumPy cosine; hosted BM25 with semantic model calls disabled
Local default models	qwen3:0.6b, embeddinggemma, translategemma:4b
Chat languages	English, Hindi, Marathi, Gujarati and Auto
Declared route handlers	60, excluding generated docs and static mounts
Interface	Public landing at /, farm dashboard at /index.html, chat at /chat.html
Database integrity	quick_check returned ok for all 6 inspected SQLite files

The vector count differs from the chunk count because a content/model cache key can be shared by identical embedding inputs. This is valid deduplication, not a missing vector. The active corpus is not the entire upstream repository.

In interviews, describe integration of pretrained models and implementation of the retrieval/application layer. Do not claim you trained Qwen, used FAISS/Chroma, built production payment processing, or mathematically eliminated hallucination. Say precisely which parts you personally implemented and which were team contributions.

2 Problem statement and requirements

Farmers face fragmented information across long PDFs, regional notices and scheme portals. A keyword match may return funding information when the question asks for required documents. A GR from one state/year may not apply to another. Scanned sources add OCR mistakes, and CPU-only inference limits long prompts and generation.

The application therefore needs source-aware retrieval, accurate citations, explicit failure states, four-language support and a simple interface. It must preserve restrictions such as dates and negation. It must distinguish model timeout from missing evidence and related evidence from a verified answer.

Functional requirements include asking agricultural questions, searching farming-only sources, inspecting evidence, choosing an answer language, maintaining farmer profiles, authenticating private operations, organizing separate chats, using speech controls, and accessing supporting farm/community/listing tools. Nonfunctional requirements include responsiveness, recoverability, privacy, maintainability and bounded resource use.

The scope is an information and demonstration platform. Current limitations include no authoritative eligibility adjudication, no trained disease classifier or price forecaster, no payment settlement/courier integration, no guaranteed source freshness and no complete cross-device chat synchronization.

3 Technology stack and selection rationale

Layer	Technology	Actual role and tradeoff
Interface	HTML5, CSS3, vanilla JavaScript	Lightweight multipage app without React/Angular build infrastructure
Browser APIs	Fetch, AbortController, localStorage, speech APIs	Requests, cancel waiting, local state and voice; platform support varies
API framework	FastAPI 0.133.1	Routes, dependencies and OpenAPI generation
Server	Uvicorn 0.41.0	ASGI process serving API and frontend
Validation	Pydantic 2.12.5	Typed request models and validators
ORM	SQLAlchemy 2.0.47	Application records, sessions and transactions
Persistence	SQLite	Simple local operation; WAL improves read/write coexistence
Optional database	PostgreSQL and psycopg2-binary	DATABASE_URL support for the ORM store; separate knowledge DBs remain SQLite
Dense search	NumPy 2.4.4	Float32 matrix and exact cosine similarity
Sparse search	Custom BM25	Inverted postings, IDF and document length normalization
Model runtime	Ollama / Groq	Local embedding/selection/translation; hosted Groq selection and translation
Cloud demonstration	Render Free	Same-origin FastAPI API and static frontend, HTTPS, automatic Git deployment
Cloud data	Supabase Free PostgreSQL	Durable app records and session digests via IPv4 session pooler
Password hashing	bcrypt 5.0.0	Salted hashes with SHA-256 preprocessing and legacy migration
HTTP clients	Requests and HTTPX	Provider/feed helpers; many paths remain synchronous
PDF extraction	pypdf 6.10.2	Page parsing and native text extraction
Scanned PDFs	PyMuPDF 1.27.2.3	Page rasterization for OCR
OCR	Tesseract and language data	Text recognition through a separate executable
Image processing	Pillow 12.1.1	Validation, orientation and preprocessing
HTML parsing	BeautifulSoup 4.14.3	Curated local HTML section extraction
Configuration	python-dotenv 1.2.1	Environment-based configuration
Tests	unittest, mocks, FastAPI TestClient	Deterministic regressions and temporary database API checks
External integrations	Open-Meteo, Twilio, Google Translation, data.gov.in	Configuration/network-dependent weather, OTP, translation and market data

requirements.txt includes transitive and compatibility packages. Their presence does not prove active use. For example, the core OCR code calls Tesseract through subprocess even though pytesseract is installed. The active system does not use LangChain, LlamaIndex, a vector DB service, JWT sessions, Redis or a neural crop-image classifier.

4 Architecture and request lifecycle

4.1 Component boundaries

The browser sends JSON or multipart requests to FastAPI. Dependencies validate identity and data. Route handlers call retrieval, database, domain helpers or integrations. SQLAlchemy manages users and business records. Independent SQLite files persist GRs, extracted pages, vectors and translations; sessions use local SQLite or the hosted PostgreSQL engine.

Ollama runs separately, normally on localhost port 11434. Uvicorn normally listens on localhost port 8000. The API calls models by HTTP rather than loading their weights directly inside FastAPI. A Graphify graph used during development maps code relationships; it is not the chatbot's production knowledge graph. This is not an active GraphRAG implementation.

4.2 Preparation versus online retrieval

Preparation validates sources, extracts page text, chunks content, computes document embeddings and persists them. Online retrieval embeds one question, searches indexes and selects a small context. The entire corpus is not placed in every prompt. This separation allows a collection larger than the model context window.

Corpus loading and in-memory index construction currently happen during module initialization. This improves later request speed but increases startup time and makes importing backend.main in tests expensive. Explicit lifecycle initialization and injected test corpora would improve isolation.

4.3 End to end question sequence

1. The browser submits question, language and an optional user ID to POST /ask.
2. Authorization verifies any account claim against a valid session.
3. Pydantic strips/validates text and restricts the language.
4. Auto language detection chooses a supported language when requested.
5. Greetings, exact reviewed questions and clarification cases take deterministic paths.
6. BM25 and semantic search generate candidates for other questions.
7. Fusion and intent/topic/state/year/identifier checks narrow evidence.
8. The local model selects a short passage or abstains; the server validates it.
9. Translation or a stored content-hash-validated localization produces display text.
10. The API returns original text, citations, evidence, warnings and timings.
11. The browser renders and locally saves the current conversation.

5 Repository responsibilities

File or group	Responsibility
backend/main.py	Routes, startup wiring, integrations and response orchestration
backend/database.py	Engines, sessions, SQLite pragmas and legacy columns
backend/models.py	16 ORM entities
backend/security.py and passwords.py	Opaque sessions, authorization, rate limits and password verification
backend/data_loader.py	Corpus composition and reviewed references
backend/pdf_ingestion.py	Page extraction, OCR caching and overlapping chunks
backend/import_maharashtra_grs.py	Preview/apply GR text imports
backend/gr_scope.py	Farming subject gate
backend/retriever.py	Unicode tokenization, BM25, exact lookup, cache and citations
backend/rag.py	Hybrid retrieval, constrained model selection and fallback
backend/retrieval_policy.py	Intent, topic, source and year checks
backend/multilingual.py	Native/Romanized aliases and search vocabulary
backend/vector_store.py	Model-aware vector reuse, SQLite storage and cosine search
backend/translation.py	Local/API translation, validation and caching
backend/document_processing.py	Bounded user upload extraction
backend/intelligence.py	Rule-based helper modules and unavailable predictive outputs
backend/setup_rag.py and build_vector_index.py	Model/OCR setup and explicit vector preparation
backend/build_knowledge_base.py	PDF-to-HTML export utility; active corpus uses direct PDF ingestion
frontend/index.html	Home landing page
frontend/chat.html	Dedicated conversation page
frontend/chat-workspace.js and CSS	Separate threads, sidebar, drafts and restoration
frontend/script.js and chat.css	Chat request/rendering, uploads and speech
frontend/ui.js	Shared navigation/auth display/location utilities
frontend/api-config.js	API origin and same-API credential handling
frontend/farmer-pages.js	Page-specific operational interactions
frontend/auth.js	Login plus environment-selected basic registration or legacy OTP screens
frontend/landing.html and landing.css	Public, responsive product introduction and app entry links
frontend/mobile.css	Shared mobile navigation, cards, forms and chat viewport fixes
backend/deployment_knowledge.py	Allowlisted GR/OCR packaging and verified cloud restoration
frontend/community.js and schemes.js	Focused feature interactions

File or group	Responsibility
frontend/shop-catalog.js	Client catalog configuration
tests	Automated source regressions
launchers and render.yaml	Local startup and deployment template

The parent launcher delegates to the application launcher, so both are useful. The virtual environment is excluded from version control and should be regenerated from dependency files when moving machines. Original source data and databases are not disposable code clutter.

6 Database design and storage

6.1 Distinct persistence roles

Store	Purpose	Recovery meaning
agrogyan.db	Accounts, farms, questions, discussions, orders and operational data	Essential user/business records; preserve and back up
gr_catalog.sqlite3	GR metadata, page arrays and content hashes	Re-importable if the exact source checkout is retained
pdf_pages.sqlite3	Page extraction/OCR output and method	Rebuildable but OCR is slow and tool-sensitive
vectors.sqlite3	Float32 embeddings, active chunk metadata and model settings	Rebuildable with the corresponding corpus/model
translations.sqlite3	Provider/language/text translation cache	Regeneration increases first-response latency
sessions.sqlite3 (local) / PostgreSQL sessions (hosted)	Session digests, identity, role and expiry	Deleting signs users out
Browser localStorage	Complete chat threads, drafts and frontend preferences	Browser-local; clearing storage loses these threads

User is the account parent. A user can have a farm profile, posts, comments, query memories, orders, demand signals, subscriptions and produce listings. FarmProfile.user_id is unique, modelling a one-to-one extension. Post has many comments. Operational weather and mandi snapshots are location-oriented records, not chat messages.

OrderStatusEvent relates to OrderRecord through an order_code string, but that column is not declared as a foreign key. The code enforces that relationship by convention. Distinguish a database-enforced constraint from an application convention in interviews.

6.2 Key entity relationships

```
User 1 -------- 0..1 FarmProfile
User 1 -------- many Post
Post 1 -------- many Comment
User 1 -------- many Comment
User 1 -------- many QueryMemory
User 1 -------- many OrderRecord
User 1 -------- many FarmerProduceListing
OrderRecord.order_code -- application link -- OrderStatusEvent.order_code
```

6.3 Transactions concurrency and migrations

get_session yields a SQLAlchemy session, rolls back on exceptions and always closes it. Route handlers explicitly commit. SQLite uses WAL, synchronous=NORMAL, foreign_keys=ON and a 15-second lock timeout. WAL improves coexistence of readers and a writer; SQLite still serializes writes. A long transaction can still cause contention.

PostgreSQL has pre-ping and bounded connection pooling settings. DATABASE_URL selects the SQLAlchemy store. security.connection now also uses that engine for durable hosted sessions when PostgreSQL is selected; the local session path remains SQLite. Independent GR, OCR, vector and translation caches remain SQLite. Changing URLs does not migrate existing user records. Base.metadata.create_all creates missing tables but is not a full column migration system. ensure_legacy_columns adds selected known columns to older records. Migration failures are logged and startup continues, so a later query may still reveal a schema problem. Alembic or another versioned migration process is future work.

Many amounts, prices, coordinates and land-size fields are strings. That makes the prototype flexible but weakens numeric validation, sorting, currency arithmetic and geospatial operations. A production migration should use Decimal/numeric types, coordinates with validation and explicit units.

7 Farming only GR import and source management

7.1 Source and count definitions

The local source is orgpedia/mahGRs at recorded commit da37bd004f038185212dccc4b938f722503c9134. It supplies page-labelled extracted English and Marathi text. English text is marked machine_translation. Each record retains the government PDF link, repository path, commit and original text provenance.

The recorded import scanned 10218 matching text files. That is not the accepted farming document count. Department folders include staffing, salaries and construction. The accepted catalog has 662 unique identifiers and 1324 language records. Its raw import report counts 4096 pages and 8115 chunks across both language variants. Active retrieval preferentially avoids duplicate full-language variants and uses 2048 pages and 4239 GR chunks. Explain which count you mean before quoting a number.

7.2 Validation and conservative scope gate

The importer resolves paths and requires containment inside the repository, a supported numeric filename ending in .pdf.en.txt or .pdf.mr.txt, a size under 5 MB and sequential nonempty page markers starting at page 1. Missing, duplicate or unordered pages are rejected because reliable page citations cannot be established.

The subject is extracted before the government header. A regex gate rejects administrative/staffing/non-farming expressions before considering positive crop/farmer/irrigation/livestock terms. It does not approve a document simply because it belongs to an agriculture department. Marathi records can use a matching English subject for classification. Ambiguous relevance is rejected for review.

This is an inspectable rule-based classifier. It can miss useful documents or accept ambiguous ones, so human review and a labelled scope-evaluation set would improve it. Original rejected files remain available; an exclusion from active answers is not physical deletion.

7.3 Preview apply and idempotency

The importer previews by default. --apply writes the catalog. It reports accepted, excluded and malformed records and distinguishes created, updated and unchanged content. --all-departments expands the departments considered while retaining the farming gate. It does not disable that gate.

The catalog uses a source key, metadata JSON, page JSON and SHA-256 content hash. Repeating an unchanged import does not multiply records. Apply can remove previously imported catalog entries that now violate the scope rules, without deleting original checkout text.

7.4 Adding new GRs or PDFs

Place appropriate original PDFs below backend/knowledge_imports. Corpus loading extracts them and applies scope/source exclusions. Run python -m backend.build_vector_index and restart the backend. New content/model keys create new embeddings; unchanged keys are reused.

For repository GRs, deliberately update the local checkout, run an import preview, inspect errors/exclusions, apply approved content, rebuild vectors and restart. There is no continuous filesystem watcher or automatic GitHub synchronization service. Adding a file alone does not guarantee approved indexed content: it can be unreadable, incomplete, excluded or outside farming scope. User uploads are not automatically promoted to shared approved knowledge.

8 PDF OCR and chunking

pypdf extracts text page by page. Fewer than 20 non-whitespace characters trigger OCR consideration. Gujarat PDFs also trigger OCR if expected Gujarati script is absent. PyMuPDF renders pages at 240 DPI, Pillow prepares images and Tesseract extracts text. The PDF ingestion folder rule uses guj+eng for Gujarat and English otherwise; it is not a universal automatic Marathi scanned-PDF OCR selector.

OCR has a 35-second subprocess timeout per page. The cache stores file_hash, page, version, content and method. The version is ocr-v1. Different PDF bytes generate a different key. A future cache version should encode changes in OCR settings/tool versions as well.

The chunker targets 1400 characters with 200-character overlap, normalizes whitespace and tries to end at a word boundary after the midpoint. These are characters, not tokens. It returns text with start/end offsets. Chunks stay within a page, allowing page-specific citations. Overlap helps preserve boundary conditions but increases near-duplicate text; retrieval filters overlapping candidates.

Each chunk carries ID, source name/path, title, state, content, page, chunk and offsets where available. GRs add GR number, identifier, source URL, repository commit and provenance. Metadata must remain attached throughout retrieval and response construction.

The 13 local PDFs have 61 cached pages and yield 108 raw chunks before exclusions; 45 are active approved PDF chunks. Successful extraction does not prove OCR accuracy. Scanned text can distort dates, proper names, amounts and punctuation without a parser error.

9 Lexical retrieval and BM25

9.1 Unicode-aware tokenization

The tokenizer uses NFC normalization and case folding, removes joiners and retains Unicode letters, numbers and combining marks. Combining marks preserve Indic vowel signs/virama. Regional decimal digits become ASCII digits, improving year/amount matching. Punctuation separates words; ranking removes common English stopwords and generic request words.

The exact reviewed-question key follows a separate normalization and retains meaningful constraints. A question containing an extra year, crop or negation must not collapse to an unrestricted definition alias.

9.2 Index structures and formula

Each chunk has term frequencies, length and prepared sentence data. Document frequencies and an inverted postings map connect terms to document IDs. BM25 uses k1=1.5 and b=0.75. For each query term:

```
IDF = log(1 + (N - df + 0.5) / (df + 0.5))
term score = IDF * tf * (k1 + 1)
             / (tf + k1 * (1 - b + b * dl / avgdl))
```

N is corpus size, df is document frequency, tf is term frequency, dl is chunk length and avgdl is average length. Rare identifiers/terms contribute more than common terms. Length normalization reduces long-document advantage. BM25 cannot understand every paraphrase or prove answerability.

9.3 Reviewed exact answers

Reviewed references map questions and aliases to sourced content. An exact normalized match returns stored content and citation without model/embedding calls. This is a fast deterministic route for common well-defined questions. A score of 1.0 means exact retrieval match, not a 100 percent probability of truth or applicability.

Stored Gujarati localizations are accepted only when localization_content_hash matches the current English content hash. Updating the English answer invalidates an older localization rather than silently showing mismatched content.

10 Embeddings and vector search

An embedding is a pretrained model's numerical representation of text. The default EmbeddingGemma output has 768 components. Similar phrasing may match without shared keywords, but adjacent topics can also be similar. The model and preprocessing must match on both query and document sides; equal dimensions alone do not guarantee compatible vectors.

Document input combines title, state and content. Model-specific query/document prefixes are applied. The cache key hashes the prepared content with a model fingerprint containing model name and Ollama digest. Title/state/content/model changes trigger regeneration. SQLite stores float32 BLOBs and chunk metadata. Batches of eight new embeddings are committed so interrupted CPU builds can resume. Active chunk/index metadata is replaced after required vectors succeed.

Every vector must have finite values and nonzero norm. It is L2-normalized. Cosine similarity then equals the dot product. At request time NumPy multiplies the matrix by the normalized query, argpartition selects top candidates and a small final sort orders them. This is exact search, not ANN.

Exact scoring costs O(ND). The matrix for 4303 by 768 float32 components occupies about 12.61 MiB, excluding Python text/objects, SQLite and model RAM. A dedicated vector server is unnecessary at this scale; a larger corpus could justify ANN/HNSW with evaluation of approximate recall and maintenance cost.

A nonblocking query semaphore limits concurrent query embeddings to one per process. The default read timeout is five seconds, with a 15-second failure cooldown. Requests can use lexical retrieval instead of queueing indefinitely. A locked background worker can reconnect vectors when Ollama returns.

11 Hybrid RAG and answer grounding

Semantic retrieval takes up to 16 candidates and lexical ranking up to 32. Reciprocal rank fusion adds 1/(20+rank) for each list position. Documents present in both lists receive both contributions. This combines rank evidence without adding incompatible BM25 and cosine scales.

A heuristic reranker rewards intent and topic coverage and checks requested years. Definitions require defining text; application-document questions require actual document/certificate evidence rather than funding. Dose questions require appropriate numeric-unit evidence. Explicit states exclude other-state documents; state-independent general references can remain for general questions.

A named 18-to-20-digit GR identifier restricts candidates to that GR. Policy questions reject general educational material as a substitute for scheme evidence, while a published application checklist can be allowed. This is a heuristic reranker and scope policy, not a neural cross encoder or complete semantic reasoner.

The context default is five chunks, clamped between one and six. A conservative UTF-8 byte budget reserves space for instructions/question/profile/output. Bytes are not exact tokens. The model receives up to six short passage options within a bounded byte budget and returns an integer choice, with zero meaning no answer. Local options include temperature 0, thinking disabled, num_ctx 3072 and maximum 24 output tokens. This reduces work and unsupported prose compared with long free generation.

Ollama output streams internally so length/deadline checks can stop selection. The browser receives one completed JSON response, not token streaming. The server constructs the answer from selected source text and appends citations. The hosted Groq selector uses the same server-side source checks; it does not authorize unrestricted agricultural claims.

The validator requires valid structure, source index and quote membership. Numbers must be supported, meaningful term overlap must pass and negation must not reverse. Malformed, truncated or ungrounded output is rejected. These controls do not prove source truth, semantic entailment, current eligibility or translation fidelity. A quote can be exact but subtly irrelevant.

Mode	Interpretation
reviewed_reference	Exact sourced question/alias

Mode	Interpretation
rag	Model selected validated evidence
extractive	Deterministic source extraction
source_excerpt	Related/potential source; specific requested answer unverified
conversation	Social greeting
clarification	More input required
source_type none	No usable supported answer established

If related sources exist but the model cannot verify the fact, the UI can show explicitly labelled excerpts. That does not mean the backend has no resources. Failure/uncertain results are not cached as trusted answers. The answer LRU is protected by a lock; corpus reconstruction or vector reconnect can invalidate cache state.

12 Multilingual retrieval and translation

Four chat languages and Auto are supported. Answer language, site-wide interface translation, speech recognition and OCR language packs are separate features. The interface dropdown does not establish that every page has full Gujarati localization.

The local retrieval path preserves the original query and adds canonical farming vocabulary, native aliases and Romanized equivalents from multilingual.py. It does not require a slow local full-query translation before every search. This reduces delay and avoids losing restrictions, names, years or negation. With an official Google Translation key, an English translation may be added as search hints; the original remains available.

Gujarati script is detected by Unicode range. Hindi and Marathi share Devanagari, so a small Marathi marker-word heuristic distinguishes them; otherwise the result is Hindi. This is not a trained language identifier. Romanized input may be detected as English unless mapped vocabulary helps search. Explicit selection helps ambiguous input.

Translation has an in-memory LRU and a SQLite cache keyed by cache version, provider/model, source language, target and text. Persistent entries have a 30-day window. With TRANSLATION_PROVIDER=groq the hosted demo uses Groq. Otherwise an official Google key selects Google Translation, or local TranslateGemma is used. On Render the SQLite translation cache is disposable across restarts; the persistent app/session data is in PostgreSQL. One local translation runs at a time, text is bounded to 2500 characters, and the read timeout is 35 seconds.

Validation rejects truncation, wrong target script and changed numbers. Trailing citation markers are removed before translation and restored afterward; internal markers are checked. A short failure cooldown avoids repeated long waits. Script checks do not establish perfect Marathi/Hindi grammar, and digit preservation does not guarantee meaning. Four core reviewed entries have content-hash-validated Gujarati localizations for fast reliable wording.

With Gujarati selected, a non-Gujarati question can become a Gujarati display_question after submission. original_question remains inspectable. This is not live keyboard transliteration. When translation fails, the original is retained with a warning. First-time local translations may still be slow or awkward.

13 Frontend and conversation workspace

The public landing page is served at / (landing.html). The farm dashboard remains /index.html, login/registration is /login.html, and the dedicated conversation workspace is /chat.html. The top-bar AgroGyanGPT link opens chat.html. Its sidebar opens one conversation at a time, and New chat creates an independent thread. The conversation surface omits dashboard weather/commerce widgets. Small screens use a collapsible history menu.

Versioned localStorage is scoped to account ID or guest. Each thread stores ID, title, messages, language, draft and latest answer. Older local question history is imported as individual entries. Old plain-text history may not contain original citation links, so the UI marks that limitation. Clearing browser storage loses local threads. They are not synchronized across devices.

query_memory is a separate server history. Complete browser threads are not automatically passed to /ask as model context. A displayed history sidebar therefore does not establish multi-turn contextual reasoning. Follow-ups such as 'what about its eligibility' may need explicit nouns because the current payload is a standalone question.

Restored HTML is sanitized with a small allowlist; links are restricted to HTTP/HTTPS with safe relationships. User text is escaped before rendering. This reduces injection risk but is not encryption or immunity to same-origin XSS. New chat/switching is disabled during active requests to prevent late answers appearing in the wrong thread. Speech is cancelled when switching.

Fetch and AbortController power the request. Pending status appears immediately, a longer-wait message after eight seconds, and the browser deadline is 90 seconds. Stop waiting aborts the browser wait but does not guarantee cancellation of synchronous backend/Ollama work. The input remains editable; duplicate sends and uploads are guarded. Retry is explicit. Read aloud is user-triggered. requestAnimationFrame batches textarea resizing and scroll work. Decorative animation and unrelated page requests were reduced.

Labels, semantic headings, conversation/status roles, focus indicators and keyboard Escape help accessibility. This is not a complete accessibility certification; assistive technology, contrast, long translated answers and keyboard flows need further evaluation.

14 Authentication security and privacy

14.1 Passwords

New passwords use bcrypt over a SHA-256 hexadecimal representation, stored with a bcrypt_sha256$ prefix. SHA-256 preprocessing keeps a fixed bcrypt input length; bcrypt supplies random salt and work factor. Legacy direct bcrypt, PBKDF2-SHA256 and plain SHA-256 digest verification support migration on login. Legacy SHA-256 should be retired after migration.

Hashing is not encryption; a hash is not decrypted to recover a password. Salt stops identical passwords simply sharing the same salted output. Never include real credentials in a placement report or repository.

14.2 Sessions and authorization

The project uses opaque random sessions, not JWT. A random token is placed in the HttpOnly agro_session cookie; only its SHA-256 digest, identity, role and expiry are stored. Lifetime is 43200 seconds or 12 hours. SameSite is lax and Secure is controlled by COOKIE_SECURE for HTTPS. Cookie or Bearer identity is resolved server-side, and logout revokes it.

Claimed user IDs in path/query/body are compared to the session. A localStorage role flag is not authorization. Admin paths and order-status changes require an admin role. Community edits/deletes check ownership. User-scoped list routes require an account ID to avoid accidentally executing all-user branches.

14.3 Abuse protection and origins

The in-process sliding-window limiter permits 30 /ask calls per minute per host/path and six per minute for login, basic registration and OTP. Workers have independent counters; distributed deployment needs shared state. Proxy handling must be configured deliberately.

CORS allows configured origins plus localhost development origins with credentials. Write requests with Origin are checked. This is useful origin protection, not a dedicated CSRF token mechanism. Production testing should cover CSRF, proxy headers, content types and uploads.

14.4 OTP registration

Real SMS/WhatsApp OTP uses Twilio configuration. The database stores a code hash, pending registration payload, status, expiry, attempts and provider metadata. The pending password is hashed. Expiry is ten minutes, resend cooldown 45 seconds and maximum wrong attempts five.

The default mock provider returns 503 instead of exposing an OTP or falsely verifying a phone. Real registration therefore requires configured delivery credentials. A short OTP hash has a small brute-force domain; keyed verification/pepper, stronger abuse controls and lifecycle cleanup are future hardening. Default setting names or old manual-fallback variables are not proof that manual verification is active.

14.5 Privacy and source access

Private question feeds are filtered by authenticated account. Community publication is separate and explicit. Knowledge routes validate source IDs and path containment rather than accepting arbitrary local paths. This report includes schema and aggregate inventory, not private names, emails, phone numbers, credentials or actual user messages.

15 Supporting modules and working level

Feature	Implemented behavior	Boundary
Farm profile	Persistent user/farm fields	Missing data can use model-farm defaults
Weather	Open-Meteo helpers and cached snapshots	Needs location/network; freshness must be inspected
Mandi	Custom feed or data.gov.in adapter	Requires configuration and matching dated records
Scheme matching	Profile-based rules and stored snapshots	Not official eligibility adjudication
Planner/briefing	Templates using crop/season/soil	Not trained optimization/yield models
Pest risk	Not assessed without observations	No district outbreak prediction model
Crop doctor	Not diagnosed response	No disease classifier configured
Document insights	Extraction and keyword classification	Not expert interpretation or shared approved knowledge
Community	Posts/comments/counters/reactions	Likes are counters rather than unique voter records
Community insights	Word frequency and ranked highlights	Not verified agronomy or an LLM consensus
Shop	Catalog/inventory/orders/demand signals	No payment settlement/courier verification
Produce listings	Listing quantities and order records	Needs stronger concurrent-stock guarantees
Order timeline	Initial event and admin updates	No automatic delivered-state verification
Market alerts	Threshold subscriptions and in-app records	SMS delivery not established by a channel field
Expert support	Official ICAR KVK directory link	Not a verified booking network
Admin	Protected analytics and operations	Needs private admin setup and further production hardening

Weather uses stored coordinates or geocoded district/state; the default snapshot freshness interval is 45 minutes. Temperature, humidity, wind and precipitation probability include source information. A successful request does not guarantee accurate location or current data. Some rule-helper outputs deliberately report unavailable instead of fabricating live values.

An async HTTPX helper exists, but many synchronous routes use blocking network helpers. FastAPI runs synchronous handlers in a thread pool; ASGI does not make all database/network calls automatically nonblocking.

Shop orders currently accept some client item/total values, adjust inventory and create demand/order events. Produce totals use listing price times quantity. Before real commerce, recompute shop totals from authoritative server prices, validate all quantities, use Decimal/currency units and make stock updates atomic to prevent overselling. These are reported limitations, not changes performed during this report/cleanup task.

16 API contracts and error design

Example guest request:

```
POST /ask
{
  "question": "Which documents are required for tractor subsidy?",
  "language": "Gujarati"
}
```

Guest requests omit user_id. A signed-in user may supply only their own ID with valid session authority. Question text is stripped, must not be blank and is limited to 2000 characters. Unsupported language values are rejected.

Response field	Meaning
answer / original_answer	Display wording and retained source-language wording
display_question / original_question	Sent bubble translation and original input
answer_language / translation_status	Selected language and translation result
source_type / answer_mode	Response category and evidence verification mode
confidence / confidence_kind	Retrieval relevance, not truth probability
references	Source metadata, page, URL and provenance
evidence	Exact passages tied to citation indices
matched_documents	Matched source metadata
retrieval_ms / generation_ms	Reported stage timings on that code path
response_ms	Backend time including translation work
cache_hit	Retrieved-answer cache use
warnings	Freshness/model/translation limitations
context_chunks / retrieval_method	Context size and retrieval path

400 is bad domain input; 401 missing/invalid login; 403 forbidden; 404 absent resource; 409 conflict; 422 schema validation; 429 rate limiting; 502 provider communication failure; 503 unavailable dependency/database; 500 unexpected server failure. Errors have user-oriented messages while logs retain diagnostic context.

A 200 /health primarily indicates database connectivity and retriever readiness. Deep fields and model/translation status must be inspected separately. Unchecked availability is not confirmed availability. FastAPI supplies /docs and /openapi.json; static pages are mounted after API routes. The API appendix lists real route decorators, excluding generated docs/static URLs.

17 Algorithms and engineering concepts

RAG separates evidence retrieval from pretrained knowledge. Sparse search ranks lexical features; dense search compares embeddings. Rank fusion combines positions. Extractive answers preserve source wording; abstractive answers generate new prose. Provenance records origin. Abstention deliberately declines unsupported claims.

Dictionaries support aliases and postings. Counter stores term frequencies. Sets support coverage/numeric checks. OrderedDict implements the answer LRU under a reentrant lock. NumPy stores the float matrix. SQLite provides durability. A deque stores recent rate-limit timestamps. Semaphores bound expensive work and locks protect shared state. Cooldowns behave like a small circuit-breaker mechanism.

Index building is proportional to tokenized corpus work and posting storage. BM25 query cost depends on postings/candidate union and ranking. Exact vector scoring is O(ND); top-k partition is approximately O(N), followed by a small sort. Exact alias lookup is average O(1) after proportional text normalization. Content hashing is proportional to input length.

Do not call the entire request O(1) because a dictionary or cache lookup is constant-time. Inference, translation, I/O, normalization and validation remain part of latency. FastAPI database/auth dependencies and injected test doubles demonstrate dependency injection. The app is a modular monolith with a separate local model service, not a microservice architecture merely because it has multiple databases.

18 Performance and evaluation

Query embedding, sparse/dense ranking, filtering, passage selection, answer translation and sent-question translation contribute to delay. On the described i5/32 GB CPU-oriented laptop, model work and uncached translation usually dominate the small matrix search. Model warm-up and startup must be measured separately.

Reviewed exact answers bypass model work. Persisted embeddings reuse unchanged hashes. Query/answer/translation caches reduce repeats. Bounded context/output, concurrency guards, timeouts and cooldowns prevent repeated expensive failures. UI progress/cancel controls improve control without guaranteeing backend cancellation.

Historical local examples showed common English reviewed answers around 0.01 to 0.02 seconds; cached/stored Gujarati examples around 0.01 to 0.03 seconds; a Maharashtra drip-scheme query around 2.23 seconds; first-time Hindi/Marathi soil translation around 8 to 10 seconds; and a Gujarati checklist translation around 24 seconds before its stored localization. They are isolated observations with different cache states, not p50/p95, throughput, accuracy or SLA evidence.

A proper evaluation needs labelled answerable/unanswerable/ambiguous/state/year/adversarial queries in all four languages. Separate cold startup, cold/warm models and caches. Record p50/p95 latency, failures, mode and stage timings at controlled concurrency. Retrieval metrics include Recall@k, Precision@k, MRR and nDCG. Answer evaluation needs citation correctness, support, completeness and abstention quality. Translation requires proficient speaker checks of meaning, negation, dates, amounts and naturalness. Similarity scores are not accuracy metrics.

19 Testing and verification

Existing test sources cover exact extraction/citations, unsupported doses/numbers, definition intent, multilingual tokenization/aliases, GR page validation, source scope, hybrid ranking, vector reuse, grounding, translation caching, preserved numbers/citations, uploads and API behavior. Models/providers are often mocked so deterministic regressions do not need paid services.

A unit test checks a small function; an integration test checks a route/database interaction; a live smoke test checks the actual server/models/browser. Mocked tests cannot establish that Twilio credentials work, OCR language packs exist or a government PDF is current. Previous work recorded 95 passing tests; the current run/count is recorded separately in the verification appendix. Only present .py files count as test source, not stale bytecode caches.

```
python -m unittest discover -s tests -v
node --check frontend/script.js
node --check frontend/ui.js
node --check frontend/chat-workspace.js
```

The suite configures temporary application/session databases. Importing main still loads the corpus and can make discovery expensive. A future refactor should isolate pure unit tests from local-source integration tests and inject a lightweight corpus.

Manual checks should include Home-to-chat navigation, new/restored isolated conversations, source links, four-language examples, an unsupported exact dose, a mismatched state/year, cancel/retry, denied private access, and explicit unavailability for unconfigured providers. Passing a finite suite is not proof of complete security or agricultural correctness.

20 Configuration startup and maintenance

The launcher describes Python 3.10 or later; the inspected development environment uses Python 3.12. Install requirements in an isolated environment, install Tesseract and language packs, run Ollama, prepare models/OCR and build embeddings before starting the API.

```
python -m venv venv
venv\Scripts\activate
python -m pip install -r requirements.txt
python -m backend.setup_rag
python -m backend.build_vector_index
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

The setup helper prepares the embedding and answer models plus OCR packs. It does not pull the separate translation model; run ollama pull translategemma:4b explicitly for local regional translation. Setup may download large models. Preparation is not expected on each question. Inspect the actual installed models and never commit private .env keys.

Setting	Purpose
DATABASE_URL	ORM application database
LLM_PROVIDER	Ollama default; optional configured cloud selector
OLLAMA_URL / OLLAMA_MODEL	Model service and selector
EMBEDDING_MODEL / TRANSLATION_MODEL	Separate embedding and translation models
VECTOR_STORE_PATH / GR_CATALOG_PATH	Knowledge-store paths
RAG_ENABLED	Hybrid/model layer toggle
RAG_MAX_CONTEXT_CHUNKS	Default 5, bounded from 1 to 6
RAG_TIMEOUT_SECONDS	Default 15-second selector deadline

Setting	Purpose
EMBEDDING_QUERY_TIMEOUT_SECONDS	Default five-second query read timeout
GOOGLE_TRANSLATE_API_KEY	Optional official translation API
OPENAI_API_KEY / GEMINI_API_KEY and LLM_MODEL	Optional cloud selector configuration
ADMIN_EMAIL / ADMIN_PASSWORD_HASH	Private admin setup
COOKIE_SECURE / CORS_ORIGINS	HTTPS cookie/origin configuration
OTP_PROVIDER and TWILIO settings	Real SMS/WhatsApp delivery
DATA_GOV_API_KEY / AGMARKNET_RESOURCE_ID	Mandi integration
MARKET_FEED_URL / SCHEME_FEED_URL	Optional external feeds
LIVE_CACHE_MINUTES	Default 45-minute snapshot interval

Use SQLite's backup API or a coordinated consistent backup. Copying only a live .db while ignoring WAL can lose committed information. Preserve source commits/provenance, preview imports, rebuild, restart and run smoke tests. Keep restoration snapshots. Import/build/startup can write caches or schema compatibility changes and are not read-only inspection operations.

The current render.yaml configures the deployed free demonstration, verified live on 24 October 2026; it is not a production uptime/security guarantee. A cloud container cannot assume the developer's local Ollama, OCR installation or durable filesystem. localhost means that container. Plan model hosting, persistent disks, HTTPS cookies, secrets, backups and suitable CPU/RAM. Multiple workers duplicate indexes and do not share in-memory locks, counters or caches. Scaling needs a consistency/resource plan.

21 Known limitations and next engineering work

The highest-priority gap is source freshness and supersession. Historical GRs need effective dates, amendment links and withdrawal tracking. Exact quotations can still be irrelevant or outdated. Multilingual heuristics and translations need a labelled four-language evaluation set. Script/number checks do not guarantee natural or faithful translation.

Full browser threads are not retrieval context on every request. Pronoun-based follow-ups need contextual query rewriting, with privacy/token safeguards. The hosted BASIC_REGISTRATION=true path does not require OTP. The optional legacy OTP flow and live mandi feed still require providers. Several AI-labelled helpers are rules/templates or unavailable responses, not trained prediction. Module status labels such as scaffolded do not implement subscriptions, telephony, billing or offline synchronization.

Before real commerce, validate quantities, recompute totals server-side, use Decimal and atomic stock updates, and integrate verified payment/fulfillment. Other improvements include splitting main.py into routers/services, lifecycle initialization, versioned migrations, typed fields, shared rate limiting, request IDs, dependency metrics and meaningful load/security tests.

Some existing code merits focused security review. Public APIs, path/query identity checks, upload bounds and redundant frontend metadata should be tested against the actual threat model. The report inventories implementation; it does not certify it as production secure. Unrelated fixes were not silently bundled into the cleanup task.

22 Cleanup performed

The cleanup removed 105 audited disposable files, reclaiming approximately 3.5 MiB. These were caches, superseded server/test logs, one-off audit/benchmark/shutdown scripts, diagnostic JSON/image previews, an unreferenced bilingual preview and a duplicate short startup note. Setup content is covered by README and this guide.

Application databases and sidecars, user records, backups, original PDFs, exclusion/subject-review provenance, the GR checkout, OCR data, environment, tests, code and deployment files were preserved. The Graphify graph and metadata were retained because they are used to navigate the codebase.

cleanup-manifest.json records each removed path, size, hash and reason. A hash is an audit fingerprint, not a backup. Regenerable outputs can be recreated, but deleted one-off helper scripts are not implied to be recoverable merely because a hash exists. Exact bytes are in the appendix. Subsequent verification may regenerate Python bytecode caches normally.

23 Placement introduction and resume

23.1 Thirty second introduction

I built AgroGyanGPT, a multilingual agricultural information application using FastAPI, SQLAlchemy and a lightweight JavaScript frontend. It retrieves farming government resolutions and sourced references with source citations. The local setup supports hybrid BM25/vector search with Ollama; the deployed free demo uses BM25 and Groq, with Supabase PostgreSQL for durable accounts and sessions. My focus was evidence grounding, multilingual fallbacks, mobile usability and a repeatable GitHub-to-Render deployment workflow.

Use I only for work you personally performed and understand. For team work use we and specify your contribution. Do not invent users, traffic, revenue, accuracy or business impact.

23.2 Two minute walkthrough

The problem was fragmented farmer information in PDFs and regional languages. I prepared knowledge offline by validating farming relevance and page markers, extracting text/OCR, creating overlapping chunks and storing content/model-aware embeddings. At request time BM25 captures exact scheme names and dense search helps paraphrases. Rank fusion and intent/state/year checks narrow candidates.

Instead of unrestricted generation, the small model selects bounded source evidence. The server validates it, attaches citations and translates display text. If evidence or models are insufficient, the app labels excerpts or abstains. The UI separates Home from a dedicated chat with independent browser-saved threads. Supporting modules include profiles, community and prototype commerce, with current limitations clearly identified.

The central tradeoff is CPU speed versus model/language quality. Caching, reviewed fast paths, small prompts and concurrency controls help, but uncached translation remains slower. My next priorities would be multilingual evaluation, source-supersession tracking, test/lifecycle isolation and production hardening.

23.3 Resume bullets

- Built a FastAPI agricultural information platform with four-language chat, SQLite/SQLAlchemy persistence and a dedicated conversation interface.
- Implemented hybrid BM25 and embedding retrieval over 4303 source passages with GR provenance and constrained local evidence selection.
- Added content/model-aware embedding reuse, translation caching, bounded inference and explicit abstention/fallback behavior.
- Developed regression coverage for intent, citations, multilingual text, GR source validation and translation integrity.
- Deployed a same-origin HTTPS demo on Render with Supabase session-pooler connectivity, cloud evidence selection and basic password registration.
- Added a public landing page and a shared accessible mobile menu; checked narrow layouts across eleven signed-in pages.

The 19 October baseline passed 95 tests; the 24 October suite includes registration/cloud regressions and passed 102 tests before the current landing update. A test count is not a percentage accuracy measurement. A valid accuracy claim needs a dataset, baseline, metric and measured result.

24 Interview questions and answer guidance

24.1 Why RAG instead of fine tuning

RAG retrieves inspectable sources and updates knowledge by importing documents rather than retraining. Fine tuning can shape task behavior/style but does not automatically provide fresh facts or page citations. I integrated pretrained models and implemented retrieval; I did not fine tune Qwen or EmbeddingGemma.

24.2 What database do you use

Local application records use SQLite through SQLAlchemy; the hosted demo uses Supabase PostgreSQL for app records and opaque session digests. GR/OCR bundles remain SQLite. Local embeddings use a SQLite cache and exact NumPy cosine search. The free cloud demo disables semantic calls and uses BM25 with Groq evidence selection. There is no hosted vector database or MongoDB component.

24.3 BM25 versus embeddings

BM25 ranks shared terms using frequency, rarity and length normalization; exact names/IDs are strengths. Embeddings compare learned numerical representations and help paraphrases. Both can fail, so hybrid retrieval combines their candidate ranks and then applies evidence policies.

24.4 Explain rank fusion

Each ranked list contributes 1/(20+rank) per document. Documents present in both lists gain both contributions. This avoids directly combining scores with incompatible scales. Candidate limits and constants should be tuned against labelled questions.

24.5 Why chunk overlap

Conditions can cross boundaries. A 200-character overlap preserves local context around a 1400-character target chunk. More overlap adds duplicates/storage; larger chunks consume context; smaller chunks can lose qualifications. Page-aware boundaries preserve citations.

24.6 How do you prevent hallucination

I reduce unsupported answers using bounded passage selection, source validation, quote membership, number/negation checks and citations. I return excerpts or abstain when support is insufficient. These checks are not a proof of truth, relevance or current policy applicability.

24.7 Does the model have all documents in context

No. Indexes hold the corpus and retrieval chooses a few relevant chunks. The pretrained model's weights and the application's knowledge store are separate. More documents only help when they are relevant, indexed and ranked well.

24.8 How does regional retrieval work

Unicode tokenization preserves Indic words. Native/Romanized aliases add canonical vocabulary and dense retrieval adds semantic signals. Optional API translation can add hints. The local retrieval path preserves the original and does not wait for mandatory local English translation.

24.9 What if Ollama stops

Reviewed answers and lexical extraction still work. Semantic selection/translation can become unavailable and responses must say so. Short query timeouts, cooldowns and background reconnection reduce repeated delay. A cache hit does not prove a model is currently running.

24.10 Are confidence scores probabilities

No, they represent retrieval relevance. An exact match can score 1.0 while source freshness/applicability still needs checking. Calibration requires a labelled dataset and correctness evaluation.

24.11 How are new files indexed

They must pass ingestion/scope checks, enter the corpus and be included in an explicit vector build/restart. Changed content/model hashes produce new vectors; unchanged keys are reused. There is no continuous automatic watcher.

24.12 Why SQLite and how would you scale

SQLite minimizes local operational overhead. WAL helps readers/writers, but concurrent writes remain limited. At larger scale I would measure first, then consider PostgreSQL, shared caches/limits and ANN search. Changing DATABASE_URL does not copy local user data or independent knowledge stores. Hosted sessions now use the configured PostgreSQL engine; local sessions remain SQLite.

24.13 How are passwords and sessions protected

New passwords use salted bcrypt with SHA-256 preprocessing. Sessions are random opaque tokens with server-stored digests and HttpOnly cookies. Roles/identity are checked server-side, not trusted from browser flags or editable IDs.

24.14 Sync versus async in FastAPI

FastAPI supports both; synchronous handlers run in a thread pool. A blocking network call inside the project remains blocking work even if ASGI is used. An async helper existing in a file does not make every route asynchronous.

24.15 What is your biggest limitation

Freshness and semantic relevance: a quoted historical passage can fail to answer a current question. Multilingual quality, heuristic language detection and CPU translation latency also matter. I would prioritize labelled evaluation and supersession metadata.

24.16 Is crop diagnosis a trained AI model

No configured disease classifier exists. OCR extracts text; crop-doctor currently reports not diagnosed. A future model needs suitable datasets, field-condition evaluation and safe limits. I should not present it as a CNN or vision model.

24.17 How did you improve the UI

I separated chat from dashboard widgets, reduced animations/background requests and batched resizing/scrolling. Progress, stop waiting and retry improve control. Rendering and model latency need separate measurement.

24.18 Is sidebar history model memory

No. Browser threads restore UI state but /ask does not automatically receive the whole conversation. Server query history is separate. Multi-turn context rewriting is future work and must preserve constraints/privacy.

24.19 How would multiple workers behave

They duplicate the corpus/matrix and process-local caches, semaphores and rate counters. They can increase RAM or database contention. Scaling requires coordination and measured bottleneck analysis, not just increasing workers.

24.20 What proves the project works

Deterministic tests cover specific behaviors; live smoke checks cover real dependencies; browser checks cover UI flows. A strong evaluation also needs labelled retrieval/answer/translation quality and load measurements. Finite passing tests are evidence within scope, not total correctness.

25 Demonstration and troubleshooting

A seven-minute demo can start with the public landing page, move to the farm dashboard, open AgroGyanGPT, ask a basic sourced definition, inspect a MahaDBT checklist source, demonstrate Gujarati and original wording, ask a page-specific GR question, show an honest abstention, and finish with New chat/sidebar and architecture. Warm required models and label cache use rather than representing it as cold latency.

Symptom	Likely reason	Check
Connection refused	Server stopped/wrong port	Launcher and port 8000
Health ready but vectors unavailable	Lexical ready, embeddings failed	vector_index and model installation
Related topic but no answer	Requested fact missing or intent/scope restriction	Evidence and answer mode
Regional reply remains English	Translation failed validation/service	Status, warnings and model
First response slow	Cold model or uncached translation	Stage times and cache state
OTP not sent	Optional legacy flow selected	Hosted basic registration needs no SMS; check BASIC_REGISTRATION
Cloud database network unreachable	Direct IPv6 host used	Supabase Session pooler, port 5432
Password authentication failed	Wrong database credentials or URL encoding	Private Render DATABASE_URL
Registration settings could not load	API script order or wrong origin	Current api-config.js initialization before auth.js
Phone shows old UI	Previous deployment or cached document	Match deployed commit; reload without clearing local chats
No mandi prices	Missing key/feed or matching data	Feed status/source timestamp
Added PDF not answering	Import/build/restart missing or exclusion	Ingestion/scope report
OCR unavailable	Missing executable/language/clear text	Setup and Tesseract language listing
History absent on another device	Browser-local storage	Same browser/account
SQLite locked	Write contention/long transaction	WAL, transaction duration

26 Glossary and revision checklist

RAG means retrieval augmented generation. Grounding connects output to evidence; provenance tracks origin; hallucination is unsupported generated content; abstention declines an unsupported answer. OCR recognizes text in images. Tokens differ from words/characters. An embedding is a numerical text representation; cosine measures direction similarity. BM25 is lexical ranking; an inverted index maps terms to documents. Top k means the highest-ranked k candidates.

Precision is the relevant fraction of returned results; recall is the retrieved fraction of relevant results. MRR measures reciprocal position of the first relevant result; nDCG accounts for graded relevance and order. LRU evicts least recently used cached results. Hashes fingerprint content; salt randomizes password hashes. A semaphore limits concurrency; a lock protects shared state; a deque supports sliding request windows.

Authentication proves identity; authorization checks access. HttpOnly limits JavaScript cookie reads; SameSite controls cross-site cookie sending; CORS controls browser cross-origin access. CSRF abuses browser authority; XSS injects script into an origin. Transactions commit or roll back related writes. Foreign keys enforce relationships. ORM maps rows to objects. WAL allows SQLite readers and a writer to coexist better. Idempotency avoids duplicate effects on repetition.

Be able to draw both ingestion and query paths, explain the exact database roles and counting levels, derive BM25/cosine/fusion at a high level and defend chunk parameters. Know why an exact quote can still be irrelevant, why freshness differs from similarity and why browser history differs from model memory. Practise your own contribution honestly. Do not memorize claims that the code does not support.

27 Evidence and references

Primary evidence is the inspected repository: main.py, rag.py, retriever.py, vector_store.py, pdf_ingestion.py, import_maharashtra_grs.py, multilingual.py, translation.py, security.py, passwords.py, models.py and chat-workspace.js. The appendices identify actual code locations. technical-evidence.json and cleanup-manifest.json preserve the aggregate inventory and cleanup audit.

The imported repository is https://github.com/orgpedia/mahGRs. Government PDF and repository-commit links are stored per record. Reviewed entries retain official source URLs in backend/knowledge/farming_reference.json; the soil source is in backend/knowledge/states/soil_basics.html. External URLs are source/further-reading locations, not a claim that policy currency or provider availability was independently verified during this task.

28 Deployed free demonstration architecture

28.1 Deployment topology and environment differences

The demonstration is available at https://agrogyan-demo.onrender.com/. The browser loads the frontend and API from the same HTTPS origin. Render runs one Uvicorn worker, reads the packaged knowledge corpus, constructs BM25 postings at startup, and connects to Supabase PostgreSQL through its IPv4 Session pooler. Groq receives the question and bounded retrieved passages for evidence selection, and supplied text for requested translation. Do not describe this as fully offline or claim that questions never leave the server in cloud mode.

```
Phone / desktop browser
        | same-origin HTTPS
        v
Render Free: FastAPI + static frontend + BM25
        |-- Supabase Session pooler (IPv4 + SSL) -> PostgreSQL app/session records
        |-- Groq Free -> bounded evidence selection and translation
        |-- packaged public GR/OCR SQLite files -> page-aware source text
        `-- browser localStorage -> per-browser conversation sidebar
```

Concern	Local setup	Free hosted demonstration
App data	SQLite agrogyan.db	Supabase PostgreSQL
Sessions	sessions.sqlite3	PostgreSQL sessions table via engine.begin
Retrieval	BM25 plus optional exact cosine vectors	BM25 and multilingual query aliases
Embeddings	EmbeddingGemma via Ollama	SEMANTIC_ENABLED=false; local vectors not queried
Answer selection	qwen3:0.6b via Ollama	Groq openai/gpt-oss-120b, environment-selectable
Translation	TranslateGemma or configured Google API	Groq with script/number/citation validation
Registration	Configurable OTP/basic flow	BASIC_REGISTRATION=true, password registration
Persistent filesystem	PC storage	Render filesystem is disposable
HTTPS cookie	Local development configuration	COOKIE_SECURE=true
Hosting trigger	Start local process	Push main -> Render automatic build/deploy

This is a cost/resource tradeoff. The free cloud profile does not silently retain vector search or local model weights. Cloud model support and quotas may change. The local hybrid implementation remains available and unchanged by the hosted configuration.

28.2 Durable data and public knowledge packaging

backend/deployment_knowledge.py reconstructs only the gr_documents and pages tables from the two allowlisted local knowledge stores. It excludes extra tables and deleted SQLite pages. It compresses each reconstructed database with gzip and records SHA-256 integrity hashes, row counts and uncompressed sizes in deploy/knowledge/manifest.json. The current bundle contains 1324 GR language records and 61 cached PDF pages. These are not the same as 662 unique GR identifiers or 4303 active retrieval chunks.

Render restores the package during its build. Unpack verifies hashes and refuses to overwrite an existing database. App records, passwords, session digests, user uploads and translation histories are not in the public package. This solves the earlier empty-corpus risk when ignored local knowledge databases were absent from a GitHub checkout.

A separate Supabase project stores new cloud demo accounts and server records. Existing PC user records were not automatically migrated or uploaded. In PostgreSQL mode, security.connection uses an adapter around SQLAlchemy engine.begin; positional SQLite placeholders are translated into named bound parameters, and sessions live in a separate

PostgreSQL sessions table. The 16 ORM tables do not include this manually created session table. In local mode the existing SQLite session implementation remains.

The Supabase Data API was disabled during project setup because these tables are accessed by the backend, not anonymous browser client libraries. Public Data API exposure without suitable row-level security would be a different security model and is not required here. The database URL and Groq key belong only in private Render Environment settings.

28.3 Free-tier constraints and operational choices

Render can sleep after inactivity, causing a cold first visit. Supabase Free can pause inactive projects. Groq enforces free request/token quotas. Free-plan limits are not an uptime guarantee and can change; no application code deliberately upgrades a plan or switches to paid inference. Keep payment methods and paid upgrades disabled if zero billing is a requirement. A custom purchased domain is unnecessary because the Render HTTPS subdomain works on phones.

No local PC needs to remain on for the hosted demo. Conversely, the local app still needs its own server and model configuration. A phone opening localhost would contact the phone itself, not the developer PC. The shared frontend uses the public origin when hosted.

Live OCR of new uploads still requires a Tesseract executable, which the current Python Render setup does not provision. Bundled PDF OCR text is already available. Real SMS delivery, payment settlement, a trained crop-disease classifier and a trained mandi forecaster were not added by deploying the app.

28.4 Failures diagnosed during deployment

Observed failure	Mechanism	Resolution and lesson
Direct Supabase host: Network is unreachable	Direct endpoint resolved to IPv6 unavailable to this Render service	Use the verified IPv4 Session pooler, not a guessed hostname or paid IPv4 add-on
Pooler: password authentication failed	Database credentials did not match or URI was malformed	Set correct database password/project-qualified username; percent-encode password characters; require SSL
Registration settings failed to load	auth.js ran before deferred api-config.js and captured localhost fallback	Initialize API config first; hosted fallback uses window.location.origin
Historical password errors in logs after a healthy deployment	Older failed attempts remained in log history	Check the latest commit/status and current /health rather than conflating timestamps
Phone header occupied much of screen	Stacked shared nav and competing responsive grid rules	Shared compact mobile menu and last-loaded responsive layer
Farmer Setup broke into single letters in chat	Narrow grid cells plus overflow-wrap:anywhere	Full-width mobile menu rows with normal word breaking

The database health endpoint reports connectivity and available corpus/index state. Configured model keys are not a successful model call. Live language/source tests remain necessary even when /health says ok.

29 Public landing page and navigation model

The bare GET / route now serves frontend/landing.html through FileResponse. /landing.html is also available as a static asset. The previous login and dashboard routes remain /login.html and /index.html; /chat.html remains the dedicated conversation workspace. Root-route behavior changed without changing database schemas or user records.

The new page introduces the agricultural use case, four chat languages, source-linked evidence and farm tools. It includes an actual reviewed soil example with a USDA NRCS source link, clear chatbot and sign-in/profile entry points, an explanation of the evidence-checking workflow and FAQs about coverage and browser-local history. It avoids promises of universally correct advice, live market rates without a feed or cross-device conversation sync.

The visual uses local HTML/CSS and a scalable inline SVG field illustration. No framework, JavaScript bundle, stock-video download or third-party font is required by the landing page. Semantic headings, a skip link, visible focus styles, native details/summary navigation and FAQ controls, and reduced-motion handling support accessibility. Its responsive stylesheet is independent of the app's dense legacy CSS. Chat remains usable as a guest; account-protected tools keep their existing authorization.

The app's More menu links back to About AgroGyan, and the login page links back to the public landing page. The farm dashboard continues to be the Home link inside the application. In interviews, distinguish public marketing/navigation entry from the authenticated or profile-aware application dashboard.

30 Mobile remediation across the application

frontend/mobile.css is loaded after page-specific styles on all existing application HTML pages. renderUnifiedHeader in ui.js inserts a shared mobile menu button with aria-controls, aria-expanded and an accessible label. The collapsed header is 64 pixels high. Opening the menu reveals normal-width navigation, grouped Farmer Setup/More links, language controls and account actions. Escape returns focus to the toggle; outside clicks collapse it. Desktop navigation remains visible while the mobile button is hidden.

The fix addressed the actual CSS cascade: earlier 760px grid rules forced small cells, arbitrary word wrapping and a tall stacked header. An early preview also exposed inherited flex-direction:column interacting with a 150px flex basis, producing huge action buttons. Setting the mobile action direction explicitly fixed it. This is a useful debugging example: inspect computed layout properties instead of merely hiding horizontal overflow.

The responsive layer reduces hero headings/padding, uses compact two-column summary cards where space permits, stacks forms, constrains media, improves source wrapping, and removes some expensive decorative shadows. Inputs use readable sizes; chat input text is at least 16px to avoid iOS focus zoom. Controls include visible focus outlines and touch-sized menu rows.

Chat uses dynamic viewport height plus visualViewport resize events where supported. The usable height is stored in a CSS custom property and shared with the workspace calculation; the header remains 64px and the composer stays above the usable viewport bottom. Safe-area padding accounts for device/browser controls. Resize writes are scheduled through requestAnimationFrame. Browser emulation supports these checks but does not establish perfect behavior on every iPhone keyboard/browser version.

Eleven signed-in pages were inspected locally at 320px with no detected document horizontal overflow: My Farm, Crop Care, Market, Sell Produce, Shop, Cart, Checkout, Schemes, Community, Profile and Chat. Home/login and the shared menu were inspected separately. Additional checks used 390px, a short 390x420 chat viewport, 820px tablet width and 1280px desktop width. The short chat check placed the composer bottom at the viewport boundary. A real phone was confirmed by the user to access the hosted app before this remediation; the exact updated physical-phone rendering still merits user review.

31 Publishing future changes and maintenance workflow

The complete practical guide is deploy/UPDATING_THE_WEBSITE.md. The authoritative retained folder is C:\Users\HP\Desktop\Divyansh\Agrogyanchatgpt, with application files under agrogyan-gpt-main. The Git destination is Oneday-begin/Agrogyanchatgpt and the deployed branch is main. Avoid the earlier unrelated destination when checking remotes.

The end-to-end chain is edit/save -> local verification -> Git commit -> Push origin -> Render build/start/health succeeds -> reload the same HTTPS website on desktop and phone. Saving or committing locally alone does not publish. Render's auto-deploy was verified to trigger on the pushed mobile commit; the deployment was observed Live and the public site served the new menu. The landing update uses that same established pipeline.

Use GitHub Desktop to add/select the outer repository, verify its remote, fetch/pull upstream work before editing, review source diffs, commit, and push. Confirm the exact latest commit ID in Render Deploys. A previous green deployment does not prove the newest source is online. If auto-deploy is disabled, choose Manual Deploy > Deploy latest commit. Keep the instance Free. Existing hosting needs no second upload for phones.

For local checks, use the configured Python environment and run python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 from the app directory. Open the root landing page and test login, navigation and affected features. Backend/retrieval/auth edits require the regression suite; UI edits require responsive browser inspection. Do not change the local database URL to the cloud database merely for layout testing.

Code changes and secret changes follow different channels. Public HTML/CSS/JS/Python and documentation go to GitHub. Actual DATABASE_URL, GROQ_API_KEY and passwords go privately into Render Environment. Changing a database password requires matching Render credentials. Version asset URLs where useful and refresh after a successful deploy. Avoid clearing all phone site data because chat threads are browser-local.

Knowledge changes need preparation as well as deployment. Import only supported agriculture/farming sources, preview scope and extraction quality, apply accepted updates, rebuild local embeddings if needed, then run python -m backend.deployment_knowledge pack. Commit the public bundle, manifest and necessary public sources. Render restores those files at build time. A generic browser document upload does not automatically approve or publish knowledge. The cloud profile still uses BM25 until a separately tested semantic provider is configured.

A code rollback does not undo database changes. Prefer ordinary corrective/revert commits over force-pushing shared history. Render rollback may restore recent code, but a durable schema/data migration needs its own tested recovery plan. Open the site before an interview to allow a sleeping free instance to start; do not fabricate keep-alive traffic or claim an SLA.

32 Interview revision for the hosted version

32.1 Explain the deployment in thirty seconds

The same FastAPI service serves a static frontend and JSON APIs over HTTPS on Render Free. Supabase PostgreSQL persists accounts and session digests through an SSL session-pooler connection. To fit a small free host, the cloud profile uses BM25 and multilingual aliases rather than local embedding inference. Groq selects bounded source evidence and translates requested text, with deterministic citation and numeric checks. Public reference tables are packaged with hashes so the deployed corpus is available without uploading user data.

32.2 Did you remove RAG by disabling vectors?

No. RAG is retrieval followed by model-assisted use of retrieved evidence; retrieval can be lexical or dense. The hosted path retrieves with BM25 and constrains the model to evidence selection. The local path additionally supports vector retrieval and reciprocal-rank fusion. Do not claim semantic parity without evaluation: disabling dense search may reduce recall for paraphrases and unfamiliar vocabulary.

32.3 Why did deployment fail despite working locally?

Localhost dependency addresses, direct IPv6 endpoints, ephemeral filesystems and secrets differ from a PC environment. The real failures included an IPv6 database network error, a credential rejection and JavaScript initialization that captured the visitor's localhost. These required environment-aware connection/configuration fixes, not simply repeated rebuilds.

32.4 What does a successful deployment prove?

It proves that a specific artifact built and started sufficiently to pass platform checks. It does not prove every route, provider, language or phone interaction. We separately checked database/corpus health, password registration/login, selected cited responses, source links and responsive layouts. Unit tests mock providers and cannot prove live Groq availability or faithful agricultural translation.

32.5 Which state survives a restart?

Supabase application records and sessions are durable independently of Render. The public corpus can be restored from the committed bundle. Render-local translation/OCR changes are disposable. Browser-local chat sidebars survive a server restart in that browser but do not automatically appear on a second device. Source files live in Git history; Git history is not a user-database backup.

32.6 Can you promise zero future cost or hallucination?

No universal promise is justified. The chosen demo uses free plans without intentional paid upgrades/fallbacks, but providers can change limits or availability. Source selection, citations and validation reduce unsupported output; they do not mathematically guarantee source truth, relevance, freshness or translation meaning. Explain those boundaries and your measured evidence rather than making absolute marketing claims.

Appendix A Verification snapshots

Current revision (24 October 2026)

The landing-route revision passed the full 102-test backend suite in 5.901 seconds of unittest runner time, excluding corpus import/startup. This includes the earlier basic-registration tests and four mocked Groq/semantic-mode checks. JavaScript syntax checks passed for the changed scripts. The landing page was checked at 320px, 390px, 820px and 1440px: one h1, no detected document horizontal overflow, working internal links/assets, and native mobile navigation/FAQs. The prior mobile application update was observed Live at commit 19807ca, and a public soil answer returned source citations. The user confirmed website access from a phone. Live multilingual quality, OCR uploads without Tesseract, concurrency/load and agricultural field accuracy were not established by these checks.

The landing deployment and updated report PDF are verified separately in report-validation.json. PDF layout validation is not a software correctness test.

Historical baseline (19 October 2026)

The fresh run on 19 October 2026 completed 95 tests in 5.467 seconds with OK, after discovery/import preparation. This duration is the unittest runner duration, not total process startup. JavaScript syntax checks passed for script.js, ui.js, chat-workspace.js, community.js and farmer-pages.js. All six inspected SQLite stores returned quick_check=ok. No new real-provider OTP, paid API, payment settlement or agricultural field trial was performed.

An earlier test process stalled in the restricted execution environment before test execution. It was stopped after the replacement completed. This is separate from the passing result and does not establish that every import/startup scenario is fast. That historical test log is graphify-out/placement-verification-final.log. The current deployment checks are recorded in report-validation.json; neither log is a multilingual accuracy benchmark.

Cleanup removed 105 files and 3,664,870 bytes (3.50 MiB); 0 files were skipped. Volatile bytecode may regenerate during normal validation.

Historical local storage inventory (19 October 2026)

Store	Size MiB	Tables	Integrity
agrogyan.db	0.32	16	ok
backend/knowledge/gr_catalog.sqlite3	16.68	1	ok
backend/knowledge/pdf_pages.sqlite3	0.30	1	ok
backend/knowledge/sessions.sqlite3	0.01	1	ok
backend/knowledge/translations.sqlite3	0.02	1	ok
backend/knowledge/vectors.sqlite3	56.75	3	ok

Historical latency provenance

Historical observations below were aggregated before deleting obsolete diagnostics. Cache/model states differ and the samples are not a formal benchmark.

Question	Language	Seconds	HTTP	Mode
Tell me about drip irrigation schemes in Maharashtra	English	2.23	200	rag
What does Gujarat GR say about organic farming in 2025-26?	Gujarati	18.75	200	rag

Question	Language	Seconds	HTTP	Mode
Mati mhanje kay? [Marathi]	Marathi	9.5	200	reviewed_reference
Mitti kya hai? [Hindi]	Hindi	8.03	200	reviewed_reference
Which documents are required for tractor subsidy?	Gujarati	24.14	200	reviewed_reference
What is organic farming?	Gujarati	0.03	200	reviewed_reference
Which documents are required for tractor subsidy?	Gujarati	0.01	200	reviewed_reference

Appendix B Endpoint inventory

Current route decorators in backend/main.py; generated OpenAPI/docs and static mounts are excluded. Line numbers identify this revision.

Method and path	Handler	Line
GET /auth/config	auth_config	1325
POST /auth/register-basic	register_basic	1330
POST /register	register_user	1350
POST /auth/send-registration-otp	send_registration_otp	1359
POST /auth/verify-registration-otp	verify_registration_otp	1427
POST /login	login_user	1503
GET /admin/analytics	get_admin_analytics	1559
GET /profile/{user_id}	get_profile	1566
POST /profile/preferences	update_profile_preferences	1597
GET /farm-profile/{user_id}	get_farm_profile	1608
POST /farm-profile	save_farm_profile	1617
POST /ops/live-sync	sync_live_feeds	1634
GET /ops/feed-status	get_feed_status	1644
GET /shop/inventory	get_shop_inventory	1681
POST /admin/inventory/upsert	upsert_inventory_item	1704
POST /shop/demand-signal	create_demand_signal	1724
GET /admin/commerce-overview	get_commerce_overview	1739
POST /shop/orders	create_shop_order	1781
GET /shop/orders	list_shop_orders	1808
POST /shop/orders/status	update_shop_order_status	1819
POST /market-alerts	create_market_alert	1832
GET /market-alerts	list_market_alerts	1851
GET /produce/listings	list_farmer_produce_listings	1882
POST /produce/listings	create_farmer_produce_listing	1914
POST /produce/orders	create_produce_order	1939
GET /dashboard-overview	dashboard_overview	1978
POST /ai/loan-insurance-assistant	ai_loan_insurance_assistant	2089
GET /platform/modules	get_platform_modules	2095
GET /health	health_check	2109
POST /ask	ask_question	2139
GET /knowledge/gr/{source_id}	repository_gr_source	2220
GET /knowledge/pdf/{source_id}	knowledge_pdf	2237
GET /knowledge/sources/{source_name}	knowledge_source	2252

Method and path	Handler	Line
POST /ai/explain-simply	explain_simply	2265
POST /ai/daily-briefing	ai_daily_briefing	2272
POST /ai/farm-planner	ai_farm_planner	2278
POST /ai/scheme-eligibility	ai_scheme_eligibility	2284
POST /ai/weather-alerts	ai_weather_alerts	2290
POST /ai/market-prediction	ai_market_prediction	2296
POST /ai/pest-risk	ai_pest_risk	2302
POST /ai/map-context	ai_map_context	2308
GET /ai/chat-memory/{user_id}	ai_chat_memory	2314
GET /questions/feed	get_question_feed	2337
POST /community/create-post	create_post	2362
GET /community/posts	get_posts	2382
GET /community/insights	get_community_insights	2411
POST /community/like/{post_id}	like_post	2417
POST /community/dislike/{post_id}	dislike_post	2427
POST /community/react	react_to_post	2437
PUT /community/post	edit_post	2454
DELETE /community/post	delete_post	2467
POST /community/comment	add_comment	2480
GET /community/comments/{post_id}	get_comments	2490
POST /upload-image/	upload_image	2496
POST /ai/crop-doctor	ai_crop_doctor	2515
POST /ai/document-insights	ai_document_insights	2522
GET /	serve_landing_page	2537
GET /auth/me	current_session	2544
POST /auth/logout	logout_session	2548
GET /crop-decision	crop_decision	2553

Appendix C Request schema dictionary

Defaults and field expressions below are extracted from source. Required means the model field has no source default; validators can impose additional constraints. Optional account IDs are still authorization-checked.

QuestionRequest

Field	Annotation	Default or constraints
question	str	Field(min_length=1, max_length=2000)
language	str	'English'
user_id	Optional[int]	Field(default=None, gt=0)
explain_simply	bool	False

Validators: validate_question, validate_language.

RegisterRequest

Field	Annotation	Default or constraints
name	str	Required
email	str	Required
password	str	Field(min_length=6, max_length=72)
mobile_number	str	Required
date_of_birth	str	Required
state	str	Required
district	str	Required
preferred_language	str	'English'
otp_channel	str	'sms'
role	str	'farmer'

BasicRegisterRequest

Used by POST /auth/register-basic when enabled. Email/name are stripped and checked; passwords require 8 to 72 characters. The role is fixed to farmer and phone_verified remains false.

Field	Type	Constraint
name	str	1 to 100 characters, nonblank
email	str	3 to 254 characters, basic email pattern
password	str	8 to 72 characters

LoginRequest

Field	Annotation	Default or constraints
email	str	Required
password	str	Field(min_length=6, max_length=72)

PostCreate

Field	Annotation	Default or constraints
content	str	Required
user_id	int	Required

ReactionRequest

Field	Annotation	Default or constraints
post_id	int	Required
emoji	str	Required

PostUpdate

Field	Annotation	Default or constraints
post_id	int	Required
user_id	int	Required
content	str	Required

PostDelete

Field	Annotation	Default or constraints
post_id	int	Required
user_id	int	Required

CommentCreate

Field	Annotation	Default or constraints
post_id	int	Required
user_id	int	Required
text	str	Field(min_length=1, max_length=5000)

Validators: readable_comment.

ProfileUpdateRequest

Field	Annotation	Default or constraints
user_id	int	Required
crop_name	Optional[str]	None
land_size	Optional[str]	None
soil_type	Optional[str]	None
season	Optional[str]	None
state	Optional[str]	None
district	Optional[str]	None
village	Optional[str]	None
preferred_language	Optional[str]	None
simple_mode	Optional[str]	None

OtpSendRequest

Inherits every field and constraint of RegisterRequest unchanged; no additional fields are declared.

OtpVerifyRequest

Field	Annotation	Default or constraints
phone_number	str	Required
email	str	Required
otp	str	Required
otp_channel	str	'sms'

FarmProfileRequest

Field	Annotation	Default or constraints
user_id	int	Required
farm_name	Optional[str]	None
primary_crop	Optional[str]	None
irrigation_type	Optional[str]	None
livestock	Optional[str]	None
taluka	Optional[str]	None
pin_code	Optional[str]	None
lat	Optional[str]	None
lng	Optional[str]	None

ExplainRequest

Field	Annotation	Default or constraints
text	str	Required
language	str	'English'

IntelligenceRequest

Field	Annotation	Default or constraints
user_id	Optional[int]	None
crop_name	Optional[str]	None
land_size	Optional[str]	None
soil_type	Optional[str]	None
season	Optional[str]	None
state	Optional[str]	None
district	Optional[str]	None
village	Optional[str]	None
preferred_language	Optional[str]	None
horizon	Optional[str]	'weekly'

InventoryUpdateRequest

Field	Annotation	Default or constraints
product_id	str	Required
product_name	str	Required
category	Optional[str]	None
seller	Optional[str]	None
stock_count	int	Required
unit	Optional[str]	None
price	Optional[str]	None
price_source	Optional[str]	'admin'

OrderItemRequest

Field	Annotation	Default or constraints
product_id	str	Required
name	str	Required
qty	int	Required
price	Optional[float]	0

OrderCreateRequest

Field	Annotation	Default or constraints
user_id	Optional[int]	None
customer_name	str	Required
customer_phone	Optional[str]	None
delivery_address	str	Required
payment_method	Optional[str]	'COD'
items	list[OrderItemRequest]	Required
total	float	Required

OrderStatusUpdateRequest

Field	Annotation	Default or constraints
order_code	str	Required
status	str	Required
note	Optional[str]	None

DemandSignalRequest

Field	Annotation	Default or constraints
user_id	Optional[int]	None
product_id	str	Required
product_name	Optional[str]	None
signal_type	str	Required

MarketAlertRequest

Field	Annotation	Default or constraints
user_id	Optional[int]	None
crop	str	Required
target_price	str	Required
district	Optional[str]	None
state	Optional[str]	None
channel	Optional[str]	'in_app'

ProduceListingRequest

Field	Annotation	Default or constraints
user_id	Optional[int]	None
seller_name	str	Required
seller_phone	Optional[str]	None
seller_location	Optional[str]	None
crop_name	str	Required
category	Optional[str]	'vegetable'
quantity	int	Required
unit	str	'kg'
price_per_unit	str	Required
harvest_date	Optional[str]	None
description	Optional[str]	None
image_url	Optional[str]	None

ProduceOrderRequest

Field	Annotation	Default or constraints
user_id	Optional[int]	None
buyer_name	str	Required
buyer_phone	Optional[str]	None
delivery_address	str	Required
payment_method	Optional[str]	'COD'
listing_code	str	Required
quantity	int	Required

Appendix D Complete database field dictionary

No private row contents are included. PK indicates a primary-key column. Nullable refers to the database notnull flag; SQLite integer primary keys can report notnull differently from their effective identity behavior. SQLAlchemy model declarations remain the application design reference.

agrogyan.db

users

Stored rows at inspection: 17.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
name	VARCHAR	No	-	-
email	VARCHAR	No	-	-
password	VARCHAR	No	-	-
mobile_number	TEXT	Yes	-	-
date_of_birth	TEXT	Yes	-	-
state	TEXT	Yes	-	-
district	TEXT	Yes	-	-
crop_name	TEXT	Yes	-	-
land_size	TEXT	Yes	-	-
soil_type	TEXT	Yes	-	-
season	TEXT	Yes	-	-
village	TEXT	Yes	-	-
preferred_language	TEXT	Yes	-	-
simple_mode	TEXT	Yes	-	-
role	TEXT	Yes	-	'farmer'
phone_verified	BOOLEAN	Yes	-	0
otp_channel	TEXT	Yes	-	-
created_at	DATETIME	Yes	-	-

posts

Stored rows at inspection: 58.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
content	VARCHAR	No	-	-

Column	SQL type	Nullable	PK	Default
timestamp	DATETIME	Yes	-	CURRENT_TIMESTAMP
user_id	INTEGER	Yes	-	-
likes	INTEGER	Yes	-	0
dislikes	INTEGER	Yes	-	0
reactions	TEXT	Yes	-	'{}'

Foreign keys: user_id references users.id.

query_memory

Stored rows at inspection: 58.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
user_id	INTEGER	Yes	-	-
question	VARCHAR	No	-	-
answer	VARCHAR	No	-	-
language	VARCHAR	Yes	-	-
context_tag	VARCHAR	Yes	-	-
timestamp	DATETIME	Yes	-	CURRENT_TIMESTAMP

Foreign keys: user_id references users.id.

otp_sessions

Stored rows at inspection: 0.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
phone_number	VARCHAR	No	-	-
email	VARCHAR	Yes	-	-
purpose	VARCHAR	No	-	-
channel	VARCHAR	No	-	-
otp_hash	VARCHAR	No	-	-
pending_payload	TEXT	Yes	-	-
status	VARCHAR	No	-	-
attempt_count	INTEGER	No	-	-
resend_count	INTEGER	No	-	-
provider_reference	VARCHAR	Yes	-	-

Column	SQL type	Nullable	PK	Default
provider_message	VARCHAR	Yes	-	-
expires_at	DATETIME	No	-	-
verified_at	DATETIME	Yes	-	-
created_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

farm_profiles

Stored rows at inspection: 2.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
user_id	INTEGER	No	-	-
farm_name	VARCHAR	Yes	-	-
primary_crop	VARCHAR	Yes	-	-
irrigation_type	VARCHAR	Yes	-	-
livestock	VARCHAR	Yes	-	-
taluka	VARCHAR	Yes	-	-
pin_code	VARCHAR	Yes	-	-
lat	VARCHAR	Yes	-	-
lng	VARCHAR	Yes	-	-
created_at	DATETIME	Yes	-	CURRENT_TIMESTAMP
updated_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

Foreign keys: user_id references users.id.

weather_snapshots

Stored rows at inspection: 2.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
state	VARCHAR	No	-	-
district	VARCHAR	No	-	-
condition	VARCHAR	No	-	-
temperature	VARCHAR	No	-	-
humidity	VARCHAR	Yes	-	-
wind	VARCHAR	Yes	-	-
rain_probability	VARCHAR	Yes	-	-

Column	SQL type	Nullable	PK	Default
advisory	TEXT	Yes	-	-
source	VARCHAR	Yes	-	-
captured_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

mandi_prices

Stored rows at inspection: 4.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
state	VARCHAR	No	-	-
district	VARCHAR	No	-	-
mandi_name	VARCHAR	No	-	-
crop	VARCHAR	No	-	-
variety	VARCHAR	Yes	-	-
price	VARCHAR	No	-	-
trend	VARCHAR	Yes	-	-
source	VARCHAR	Yes	-	-
captured_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

alert_records

Stored rows at inspection: 4.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
user_id	INTEGER	Yes	-	-
state	VARCHAR	Yes	-	-
district	VARCHAR	Yes	-	-
crop	VARCHAR	Yes	-	-
severity	VARCHAR	No	-	-
title	VARCHAR	No	-	-
body	TEXT	No	-	-
category	VARCHAR	No	-	-
source	VARCHAR	Yes	-	-
created_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

Foreign keys: user_id references users.id.

inventory_items

Stored rows at inspection: 7.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
product_id	VARCHAR	No	-	-
product_name	VARCHAR	No	-	-
category	VARCHAR	Yes	-	-
seller	VARCHAR	Yes	-	-
stock_count	INTEGER	No	-	-
unit	VARCHAR	Yes	-	-
price	VARCHAR	Yes	-	-
price_source	VARCHAR	Yes	-	-
source	VARCHAR	Yes	-	-
updated_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

order_records

Stored rows at inspection: 0.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
order_code	VARCHAR	No	-	-
user_id	INTEGER	Yes	-	-
customer_name	VARCHAR	No	-	-
customer_phone	VARCHAR	Yes	-	-
delivery_address	TEXT	No	-	-
payment_method	VARCHAR	Yes	-	-
item_summary	TEXT	No	-	-
item_count	INTEGER	No	-	-
total	VARCHAR	No	-	-
status	VARCHAR	No	-	-
source	VARCHAR	Yes	-	-
created_at	DATETIME	Yes	-	CURRENT_TIMESTAMP
updated_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

Foreign keys: user_id references users.id.

order_status_events

Stored rows at inspection: 0.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
order_code	VARCHAR	No	-	-
status	VARCHAR	No	-	-
note	TEXT	Yes	-	-
source	VARCHAR	Yes	-	-
created_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

demand_signals

Stored rows at inspection: 0.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
user_id	INTEGER	Yes	-	-
product_id	VARCHAR	No	-	-
product_name	VARCHAR	Yes	-	-
signal_type	VARCHAR	No	-	-
source	VARCHAR	Yes	-	-
created_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

Foreign keys: user_id references users.id.

market_alert_subscriptions

Stored rows at inspection: 0.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
user_id	INTEGER	Yes	-	-
crop	VARCHAR	No	-	-
target_price	VARCHAR	No	-	-
district	VARCHAR	Yes	-	-
state	VARCHAR	Yes	-	-
channel	VARCHAR	Yes	-	-
is_active	BOOLEAN	No	-	-

Column	SQL type	Nullable	PK	Default
created_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

Foreign keys: user_id references users.id.

scheme_snapshots

Stored rows at inspection: 3.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
scheme_name	VARCHAR	No	-	-
state	VARCHAR	Yes	-	-
crop	VARCHAR	Yes	-	-
summary	TEXT	No	-	-
eligibility	TEXT	Yes	-	-
source	VARCHAR	Yes	-	-
captured_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

farmer_produce_listings

Stored rows at inspection: 2.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
listing_code	VARCHAR	No	-	-
user_id	INTEGER	Yes	-	-
seller_name	VARCHAR	No	-	-
seller_phone	VARCHAR	Yes	-	-
seller_location	VARCHAR	Yes	-	-
crop_name	VARCHAR	No	-	-
category	VARCHAR	Yes	-	-
quantity	INTEGER	No	-	-
available_quantity	INTEGER	No	-	-
unit	VARCHAR	No	-	-
price_per_unit	VARCHAR	No	-	-
harvest_date	VARCHAR	Yes	-	-
description	TEXT	Yes	-	-
image_url	VARCHAR	Yes	-	-

Column	SQL type	Nullable	PK	Default
is_active	BOOLEAN	No	-	-
created_at	DATETIME	Yes	-	CURRENT_TIMESTAMP
updated_at	DATETIME	Yes	-	CURRENT_TIMESTAMP

Foreign keys: user_id references users.id.

comments

Stored rows at inspection: 0.

Column	SQL type	Nullable	PK	Default
id	INTEGER	No	1	-
post_id	INTEGER	No	-	-
user_id	INTEGER	No	-	-
text	TEXT	No	-	-
timestamp	DATETIME	Yes	-	CURRENT_TIMESTAMP

Foreign keys: user_id references users.id; post_id references posts.id.

backend/knowledge/gr_catalog.sqlite3

gr_documents

Stored rows at inspection: 1324.

Column	SQL type	Nullable	PK	Default
source_key	TEXT	Yes	1	-
metadata	TEXT	No	-	-
pages	TEXT	No	-	-
content_hash	TEXT	No	-	-

backend/knowledge/pdf_pages.sqlite3

pages

Stored rows at inspection: 61.

Column	SQL type	Nullable	PK	Default
file_hash	TEXT	Yes	1	-
page	INTEGER	Yes	2	-
version	TEXT	Yes	3	-
content	TEXT	Yes	-	-

Column	SQL type	Nullable	PK	Default
method	TEXT	Yes	-	-

backend/knowledge/sessions.sqlite3

sessions

Stored rows at inspection: 0.

Column	SQL type	Nullable	PK	Default
digest	TEXT	Yes	1	-
user_id	INTEGER	Yes	-	-
role	TEXT	Yes	-	-
expires	REAL	Yes	-	-

backend/knowledge/translations.sqlite3

translations

Stored rows at inspection: 21.

Column	SQL type	Nullable	PK	Default
key	TEXT	Yes	1	-
value	TEXT	No	-	-
created	REAL	No	-	-

backend/knowledge/vectors.sqlite3

embeddings

Stored rows at inspection: 4302.

Column	SQL type	Nullable	PK	Default
cache_key	TEXT	Yes	1	-
model	TEXT	No	-	-
dimension	INTEGER	No	-	-
vector	BLOB	No	-	-

chunks

Stored rows at inspection: 4303.

Column	SQL type	Nullable	PK	Default
id	TEXT	Yes	1	-

Column	SQL type	Nullable	PK	Default
cache_key	TEXT	No	-	-
metadata	TEXT	No	-	-

index_meta

Stored rows at inspection: 2.

Column	SQL type	Nullable	PK	Default
key	TEXT	Yes	1	-
value	TEXT	No	-	-

Appendix E Code symbol navigation

Current Python class/function locations from backend and tests. This is a source navigation map, not a feature-count claim.

backend/build_knowledge_base.py

- L22: normalize_text (function).
- L29: pdf_to_text (function).
- L40: text_to_sections (function).
- L45: build_html_page (function).
- L86: main (function).

backend/build_vector_index.py

- L9: main (function).

backend/data_loader.py

- L18: normalize_text (function).
- L27: slugify (function).
- L31: chunk_text (function).
- L71: load_csv_documents (function).
- L102: extract_html_sections (function).
- L155: load_html_documents (function).
- L165: load_documents (function).
- L173: load_farming_references (function).
- L195: load_corpus (function).

backend/database.py

- L37: _sqlite_pragmas (function).
- L58: get_session (function).
- L74: check_database (function).
- L113: ensure_legacy_columns (function).

backend/deployment_knowledge.py

- L16: pack (function).
- L39: unpack (function).

backend/document_processing.py

- L17: DocumentError (class).
- L21: image_text (function).
- L64: document_text (function).

backend/gr_scope.py

- L38: farming_scope (function).

backend/import_maharashtra_grs.py

- L27: parse_pages (function).
- L38: extract_subject (function).
- L45: prepare_document (function).
- L92: import_repository (function).
- L156: load_gr_documents (function).
- L191: main (function).

backend/intelligence.py

- L52: merge_profile (function).
- L69: simplify_text (function).
- L89: build_weather_overview (function).
- L93: build_market_prices (function).
- L97: market_prediction (function).
- L101: weather_alerts (function).
- L105: pest_outbreak_risk (function).
- L109: scheme_eligibility (function).
- L117: loan_insurance_assistant (function).
- L159: generate_farm_plan (function).
- L174: daily_briefing (function).
- L188: community_insights (function).
- L220: analyze_leaf_image (function).
- L224: analyze_document_text (function).
- L251: map_context (function).
- L266: build_chat_context (function).
- L278: build_dashboard_payload (function).

backend/main.py

- L109: log_requests (function).
- L120: validation_error_handler (function).
- L127: database_error_handler (function).
- L133: unhandled_error_handler (function).
- L265: hash_password (function).
- L269: verify_password (function).
- L273: is_admin_credentials (function).
- L277: is_admin_email (function).
- L281: hash_otp (function).
- L285: normalize_email (function).
- L289: now_utc (function).
- L293: is_recent (function).
- L299: safe_float (function).
- L306: coerce_price_number (function).

- L311: fetch_json (function).
- L318: fetch_json_sync (function).
- L323: generate_otp (function).
- L327: normalize_indian_phone_number (function).
- L338: phone_lookup_values (function).
- L352: build_otp_message (function).
- L362: translate_text (function).
- L381: get_user_by_id (function).
- L387: save_memory (function).
- L404: extract_crop_mentions (function).
- L409: get_timeframe_start (function).
- L419: build_admin_analytics (function).
- L538: get_or_create_farm_profile (function).
- L550: serialize_farm_profile (function).
- L565: merge_farm_profile_context (function).
- L584: ensure_inventory_seed (function).
- L588: ensure_scheme_snapshots (function).
- L622: ensure_produce_listing_seed (function).
- L626: resolve_profile_coordinates (function).
- L647: fetch_live_weather_snapshot (function).
- L688: normalize_market_rows (function).
- L720: fetch_live_market_prices (function).
- L745: replace_market_rows (function).
- L751: sync_order_timeline (function).
- L766: serialize_order (function).
- L792: refresh_live_operating_data (function).
- L860: create_market_alert_notifications (function).
- L915: decrement_inventory_for_order (function).
- L923: build_seed_weather (function).
- L937: build_seed_markets (function).
- L961: build_seed_alerts (function).
- L980: ensure_seed_operating_data (function).
- L986: send_otp_via_provider (function).
- L1056: register_verified_user (function).
- L1078: update_user_profile (function).
- L1095: QuestionRequest (class).
- L1103: validate_question (function).
- L1111: validate_language (function).
- L1117: RegisterRequest (class).
- L1130: LoginRequest (class).
- L1135: PostCreate (class).
- L1140: ReactionRequest (class).

- L1145: PostUpdate (class).
- L1151: PostDelete (class).
- L1156: CommentCreate (class).
- L1163: readable_comment (function).
- L1169: ProfileUpdateRequest (class).
- L1182: OtpSendRequest (class).
- L1186: OtpVerifyRequest (class).
- L1193: FarmProfileRequest (class).
- L1205: ExplainRequest (class).
- L1210: IntelligenceRequest (class).
- L1223: InventoryUpdateRequest (class).
- L1234: OrderItemRequest (class).
- L1241: OrderCreateRequest (class).
- L1251: OrderStatusUpdateRequest (class).
- L1257: DemandSignalRequest (class).
- L1264: MarketAlertRequest (class).
- L1273: ProduceListingRequest (class).
- L1288: ProduceOrderRequest (class).
- L1298: merge_request_profile (function).
- L1310: BasicRegisterRequest (class).
- L1317: clean_identity (function).
- L1325: auth_config (function).
- L1330: register_basic (function).
- L1350: register_user (function).
- L1359: send_registration_otp (function).
- L1427: verify_registration_otp (function).
- L1503: login_user (function).
- L1559: get_admin_analytics (function).
- L1566: get_profile (function).
- L1597: update_profile_preferences (function).
- L1608: get_farm_profile (function).
- L1617: save_farm_profile (function).
- L1634: sync_live_feeds (function).
- L1644: get_feed_status (function).
- L1681: get_shop_inventory (function).
- L1704: upsert_inventory_item (function).
- L1724: create_demand_signal (function).
- L1739: get_commerce_overview (function).
- L1781: create_shop_order (function).
- L1808: list_shop_orders (function).
- L1819: update_shop_order_status (function).
- L1832: create_market_alert (function).

- L1851: list_market_alerts (function).
- L1882: list_farmer_produce_listings (function).
- L1914: create_farmer_produce_listing (function).
- L1939: create_produce_order (function).
- L1978: dashboard_overview (function).
- L2089: ai_loan_insurance_assistant (function).
- L2095: get_platform_modules (function).
- L2109: health_check (function).
- L2139: ask_question (function).
- L2220: repository_gr_source (function).
- L2237: knowledge_pdf (function).
- L2252: knowledge_source (function).
- L2265: explain_simply (function).
- L2272: ai_daily_briefing (function).
- L2278: ai_farm_planner (function).
- L2284: ai_scheme_eligibility (function).
- L2290: ai_weather_alerts (function).
- L2296: ai_market_prediction (function).
- L2302: ai_pest_risk (function).
- L2308: ai_map_context (function).
- L2314: ai_chat_memory (function).
- L2337: get_question_feed (function).
- L2362: create_post (function).
- L2382: get_posts (function).
- L2411: get_community_insights (function).
- L2417: like_post (function).
- L2427: dislike_post (function).
- L2437: react_to_post (function).
- L2454: edit_post (function).
- L2467: delete_post (function).
- L2480: add_comment (function).
- L2490: get_comments (function).
- L2496: upload_image (function).
- L2515: ai_crop_doctor (function).
- L2522: ai_document_insights (function).
- L2537: serve_landing_page (function).
- L2544: current_session (function).
- L2548: logout_session (function).
- L2553: crop_decision (function).

backend/models.py

- L6: User (class).

- L30: Post (class).
- L42: Comment (class).
- L52: QueryMemory (class).
- L64: OtpSession (class).
- L84: FarmProfile (class).
- L101: WeatherSnapshot (class).
- L117: MandiPrice (class).
- L132: AlertRecord (class).
- L148: InventoryItem (class).
- L164: OrderRecord (class).
- L183: OrderStatusEvent (class).
- L194: DemandSignal (class).
- L206: MarketAlertSubscription (class).
- L220: SchemeSnapshot (class).
- L233: FarmerProduceListing (class).

backend/multilingual.py

- L83: normalize_digits (function).
- L87: alias_present (function).
- L101: topic_anchors (function).
- L112: has_topic (function).
- L117: search_plan (function).

backend/passwords.py

- L7: hash_password (function).
- L12: verify_password (function).

backend/pdf_ingestion.py

- L23: ocr_page (function).
- L39: overlapping_chunks (function).
- L56: load_pdf_documents (function).

backend/rag.py

- L21: RAGRetriever (class).
- L22: __init__ (function).
- L41: status (function).
- L47: _maybe_reconnect_vectors (function).
- L56: reconnect (function).
- L72: _extract (function).
- L210: _fallback (function).
- L238: _related_sources (function).
- L272: _generate (function).
- L382: _cloud_configured (function).

- L386: _cloud_select (function).
- L400: _passages (function).
- L406: _validate (function).

backend/retrieval_policy.py

- L13: intent (function).
- L28: answers_intent (function).
- L42: subject_terms (function).
- L48: topic_coverage (function).
- L55: requested_years_match (function).
- L60: policy_request (function).
- L64: source_fits_request (function).

backend/retriever.py

- L60: tokenize (function).
- L70: sentence_split (function).
- L76: _is_clean_sentence (function).
- L84: _clean (function).
- L88: OllamaClient (class).
- L95: __init__ (function).
- L102: is_available (function).
- L119: Retriever (class).
- L123: __init__ (function).
- L166: _bm25 (function).
- L177: _rank (function).
- L204: _coverage (function).
- L211: _best_sentences (function).
- L224: _references (function).
- L248: _question_key (function).
- L253: _reviewed_answer (function).
- L269: answer (function).
- L303: _extract (function).
- L393: _needs_cts_clarification (function).
- L398: _definition_target (function).
- L411: _overlaps (function).
- L418: _result (function).

backend/security.py

- L21: connection (function).
- L26: Adapter (class).
- L27: __init__ (function).
- L28: execute (function).
- L44: issue_session (function).

- L51: revoke (function).
- L56: authorize (function).

backend/setup_rag.py

- L11: main (function).

backend/translation.py

- L26: cache_connection (function).
- L34: translation_status (function).
- L38: translate_cached (function).
- L82: detect_language (function).
- L89: _normalize_digits (function).
- L93: _valid_target_script (function).
- L104: _validate_translation (function).
- L117: local_translate (function).
- L168: groq_translate (function).

backend/vector_store.py

- L26: OllamaEmbeddings (class).
- L27: __init__ (function).
- L40: embed (function).
- L63: query (function).
- L75: VectorStore (class).
- L76: __init__ (function).
- L88: embedding_text (function).
- L91: _build (function).
- L139: search (function).
- L151: status (function).

tests/test_chatbot.py

- L22: tearDownModule (function).
- L27: NoModelCalls (class).
- L31: is_available (function).
- L34: generate_json (function).
- L38: RetrievalTests (class).
- L39: setUp (function).
- L48: test_answer_is_source_text_with_matching_citations (function).
- L58: test_unsupported_doses_and_numbers_abstain (function).
- L65: test_wrong_state_is_not_supported (function).
- L69: test_live_information_abstains (function).
- L73: test_unrelated_chunks_cannot_pool_query_coverage (function).
- L76: test_cache_is_isolated_bounded_and_thread_safe (function).
- L89: test_empty_index_abstains (function).

- L92: test_greeting_is_not_a_failed_document_search (function).
- L100: test_ambiguous_land_record_question_asks_for_specifics (function).
- L108: test_definition_question_rejects_incidental_mentions (function).
- L115: test_real_corpus_soil_definition_and_exclusion_of_project_notes (function).
- L128: test_overlapping_passages_are_not_repeated (function).
- L140: ApiTests (class).
- L141: setUp (function).
- L146: test_health_and_frontend (function).
- L154: test_degraded_readiness (function).
- L164: test_validation (function).
- L170: test_anonymous_chat_has_no_invented_farm_profile (function).
- L177: test_regional_retrieval_preserves_original_without_external_translation (function).
- L184: test_failed_translation_is_visible (function).
- L190: test_gujarati_source_answer_does_not_require_retranslation (function).
- L200: test_chat_memory_persists (function).
- L215: test_explanation_preserves_doses_and_qualifiers (function).
- L220: test_source_inspection_only_serves_indexed_files (function).
- L226: test_pdf_citation_opens_registered_original_file (function).
- L234: test_comments_persist_in_database_and_reject_empty_text (function).
- L247: test_image_answer_retains_citations_and_language (function).
- L256: test_unreadable_upload_reports_ocr_failure (function).
- L263: test_blank_pdf_is_not_decoded_as_document_text (function).
- L272: test_registration_hashes_pending_password_and_clears_payload (function).
- L290: TranslationTests (class).
- L291: setUp (function).
- L302: test_network_timeout_and_success_cache (function).
- L313: test_failure_cooldown_avoids_repeated_network_waits (function).
- L334: SecurityTests (class).
- L335: setUp (function).
- L347: login (function).
- L352: test_guest_and_forged_identity_rejected (function).
- L362: test_history_is_private_and_logout_revokes (function).
- L372: test_login_rate_limit_and_bad_password_status (function).
- L377: test_manual_otp_is_not_a_verification_bypass (function).
- L382: test_old_order_does_not_auto_deliver (function).
- L389: test_produce_listing_uses_authenticated_owner (function).
- L396: GujaratiPresentationTests (class).
- L397: setUp (function).
- L402: test_selected_gujarati_localizes_greeting_and_sent_question (function).
- L412: test_auto_detects_gujarati_and_preserves_source (function).
- L421: test_unsupported_languages_removed (function).
- L425: test_translation_failure_is_not_english_answer_disguised_as_gujarati (function).

- L435: BasicRegistrationTests (class).
- L436: setUp (function).
- L441: test_password_registration_login_and_no_phone_claim (function).
- L457: test_basic_registration_disabled (function).
- L463: test_invalid_registration_and_rate_limit (function).

tests/test_cloud_demo.py

- L9: CloudDemoTests (class).
- L10: test_groq_selector_uses_only_configured_endpoint (function).
- L20: test_cloud_translation_rejects_changed_amount (function).
- L27: test_cloud_translation_keeps_citation (function).
- L33: test_disabled_semantic_search_never_initializes_ollama (function).

tests/test_gr_import.py

- L12: ImportTests (class).
- L13: setUp (function).
- L25: tearDown (function).
- L28: test_preview_does_not_write_and_import_is_idempotent (function).
- L41: test_partial_or_missing_pages_are_rejected (function).
- L49: test_long_government_identifiers_are_supported (function).
- L53: test_wrapped_subject_is_checked_before_government_header (function).
- L61: test_agriculture_department_does_not_approve_unrelated_subjects (function).
- L72: test_salted_hashes_and_legacy_verification (function).
- L80: test_original_uses_paired_subject_and_removes_previous_false_match (function).

tests/test_multilingual_rag.py

- L11: MultilingualTests (class).
- L12: test_indic_words_keep_vowels_and_virama (function).
- L18: test_offline_concepts_do_not_replace_original_constraints (function).
- L30: test_native_alias_lexical_result_can_enter_semantic_context (function).
- L43: test_selector_returns_only_exact_evidence_and_rejects_invalid_choice (function).
- L58: test_failed_generation_cannot_escape_named_source (function).
- L68: test_definition_intent_is_explained_without_discarding_question (function).
- L79: test_topic_filter_excludes_generic_budget_paragraph (function).

tests/test_rag.py

- L16: FakeEmbeddings (class).
- L20: __init__ (function).
- L23: embed (function).
- L27: query (function).
- L32: VectorTests (class).
- L33: setUp (function).
- L39: tearDown (function).

- L42: test_semantic_search_and_persistent_float_vectors (function).
- L50: test_restart_reuses_embeddings_without_regeneration (function).
- L57: test_changed_deleted_and_model_changed_sources (function).
- L69: test_failed_rebuild_preserves_committed_index (function).
- L78: test_interrupted_bulk_build_resumes_completed_embedding_batches (function).
- L84: fail_second (function).
- L98: IngestionTests (class).
- L99: test_overlap_preserves_every_character_and_bounds (function).
- L110: test_all_pages_metadata_and_ocr_cache (function).
- L127: RAGTests (class).
- L128: setUp (function).
- L138: test_model_receives_whole_retrieved_context_and_result_is_cited (function).
- L149: test_invented_citations_quotes_and_numbers_are_rejected (function).
- L160: test_model_abstention_is_not_replaced_with_general_advice (function).
- L166: test_failed_model_fallback_is_visible_and_not_cached (function).
- L173: test_timeout_preserves_cross_language_semantic_evidence (function).
- L190: test_weak_semantic_match_is_not_presented_as_source_evidence (function).
- L195: test_numbered_passage_is_mapped_to_exact_source_text (function).
- L202: test_timeout_records_actual_wait (function).
- L204: fail (function).
- L212: test_named_gr_does_not_mix_other_resolutions (function).
- L225: test_document_number_abbreviation_is_not_negation (function).
- L230: test_gr_identity_uses_metadata_but_does_not_approve_invented_amounts (function).

tests/test_retrieval_improvements.py

- L11: RetrievalImprovementTests (class).
- L12: make_rag (function).
- L17: test_reviewed_basics_and_native_aliases_need_no_model_or_embedding (function).
- L30: test_question_matching_does_not_drop_new_constraints_or_question_intent (function).
- L36: test_stored_gujarati_wording_is_tied_to_exact_source_version (function).
- L45: test_paperwork_is_not_answered_by_a_subsidy_amount (function).
- L49: test_definition_rejects_incidental_budget_text_before_model_selection (function).
- L58: test_scheme_queries_do_not_use_general_reference_as_scheme_evidence (function).
- L64: test_native_source_matches_english_topic_without_general_reference (function).
- L68: test_relevant_abstention_shows_related_sources_without_claiming_answer (function).
- L78: test_unrelated_abstention_has_no_source_claim (function).
- L86: test_lexical_hit_survives_low_semantic_score (function).
- L93: test_starting_ollama_later_reconnects_without_blocking_the_chat (function).
- L97: thread (function).
- L107: test_failed_reconnect_has_cooldown_and_preserves_keyword_index (function).
- L111: thread (function).

tests/test_translation_local.py

- L13: LocalTranslationTests (class).
- L14: setUp (function).
- L31: reply (function).
- L34: test_trailing_source_citations_are_preserved_without_model_reproduction (function).
- L40: test_internal_citations_cannot_be_lost (function).
- L45: test_changed_amount_fails_and_is_not_cached (function).
- L54: test_indic_digits_and_internal_reference_ids_are_checked_and_normalized (function).
- L60: test_marathi_script_is_not_misclassified_as_hindi_output (function).
- L65: test_explicit_source_overrides_ambiguous_devanagari_detection (function).
- L72: test_network_timeout_has_short_cooldown_but_static_greeting_still_works (function).
- L81: test_persistent_cache_avoids_repeat_model_work (function).
- L88: test_cache_is_specific_to_translation_model (function).
- L96: test_wrong_script_and_truncation_are_rejected (function).
- L104: test_static_soil_translation_does_not_discard_extra_source_sentence (function).

Appendix F File inventory and cleanup audit

Current application source/configuration files are listed below. Runtime environments, original external checkout, user databases, backups and generated report files are intentionally excluded from this source-only list.

File	Bytes
.env.example	1094
.gitignore	341
README.md	5992
backend/__init__.py	0
backend/build_knowledge_base.py	3253
backend/build_vector_index.py	508
backend/data_loader.py	10008
backend/database.py	5176
backend/deployment_knowledge.py	2488
backend/document_processing.py	4359
backend/gr_scope.py	3461
backend/import_maharashtra_grs.py	12650
backend/intelligence.py	11697
backend/knowledge/README.md	1284
backend/knowledge/farming_reference.json	24092
backend/knowledge/gr_accepted_subjects.json	398169
backend/knowledge/source_exclusions.json	945
backend/knowledge/states/gujarat.html	57191
backend/knowledge/states/maharashtra_research_units.html	8677
backend/knowledge/states/punjab.html	10126
backend/knowledge/states/soil_basics.html	787
backend/knowledge_imports/GR (guju)/1.pdf	97208
backend/knowledge_imports/GR (guju)/10.pdf	176381
backend/knowledge_imports/GR (guju)/2.pdf	78009
backend/knowledge_imports/GR (guju)/3.pdf	96119
backend/knowledge_imports/GR (guju)/4.pdf	82328
backend/knowledge_imports/GR (guju)/5.pdf	593719
backend/knowledge_imports/GR (guju)/6.pdf	159516

File	Bytes
backend/knowledge_imports/GR (guju)/7.pdf	375096
backend/knowledge_imports/GR (guju)/8.pdf	204878
backend/knowledge_imports/GR (guju)/9.pdf	220732
backend/knowledge_imports/GR (punjab)/msp.pdf	556103
backend/knowledge_imports/farming units/Farming_Research_Units_Maharashtra.pdf	4896
backend/knowledge_imports/farming units/PROB STATEMENT 2.pdf	237747
backend/main.py	102040
backend/models.py	11190
backend/multilingual.py	10134
backend/ocr/tessdata/eng.traineddata	4113088
backend/ocr/tessdata/guj.traineddata	1418394
backend/ocr/tessdata/hin.traineddata	1122751
backend/ocr/tessdata/mar.traineddata	2118233
backend/passwords.py	1274
backend/pdf_ingestion.py	5996
backend/rag.py	28775
backend/retrieval_policy.py	3613
backend/retriever.py	21252
backend/security.py	6289
backend/setup_rag.py	2173
backend/translation.py	11214
backend/vector_store.py	8031
deploy/README.md	3963
deploy/UPDATING_THE_WEBSITE.md	11561
docs/placement/build_report.py	9925
frontend/admin.html	25660
frontend/api-config.js	966
frontend/auth.js	23177
frontend/cart.html	5323
frontend/chat-workspace.css	7461
frontend/chat-workspace.js	7465

File	Bytes
frontend/chat.css	12639
frontend/chat.html	6623
frontend/checkout.html	9320
frontend/community.html	10603
frontend/community.js	31993
frontend/crop-care.html	8552
frontend/farmer-pages.js	183842
frontend/i18n.js	48402
frontend/index.html	6635
frontend/landing.css	10365
frontend/landing.html	9216
frontend/landing.js	667
frontend/login.html	11523
frontend/market.html	11881
frontend/mobile.css	8656
frontend/my-farm.html	12748
frontend/profile.html	21584
frontend/schemes.html	6269
frontend/schemes.js	10033
frontend/script.js	46510
frontend/sell-crops.html	8329
frontend/shop-catalog.js	21289
frontend/shop.html	13367
frontend/style.css	137514
frontend/ui.js	25390
render.yaml	1077
requirements-dev.txt	36
requirements.txt	604
start_agrogyan.bat	884
tests/test_chatbot.py	26661
tests/test_cloud_demo.py	2570

File	Bytes
tests/test_gr_import.py	6355
tests/test_multilingual_rag.py	6395
tests/test_rag.py	13348
tests/test_retrieval_improvements.py	7705
tests/test_translation_local.py	6449

Cleanup categories

The exact path/hash manifest is cleanup-manifest.json. Counts below summarize the original cleanup operation; they do not include ordinary caches regenerated by tests.

Category	Files	Bytes
.jpg	3	137,107
.json	22	2,323,183
.log	40	568,675
.png	3	117,186
.py	8	25,218
.pyc	28	493,369
.txt	1	132

Revision plan

Day 1: study the overview, draw the architecture, explain your contribution. Day 2: trace ingestion, BM25, vector search and one answer in code. Day 3: practise database/auth/API questions and run the demo. Day 4: revise limitations, evaluation metrics and scaling tradeoffs. Before the interview, recheck installed models, startup and a few supported questions rather than assuming cached results represent every scenario.